



中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

基于粒子系统的 Web 数字地球海洋矢量场数据 动态可视化方法

作者姓名: 刘 恒 星

指导教师: 范 湘 涛 研究员

刘 朔 副研究员

中国科学院遥感与数字地球研究所

学位类别: 理学硕士

学科专业: 地图学与地理信息系统

培养单位: 中国科学院遥感与数字地球研究所

2017 年 6 月

Method of Web Digital Earth Ocean Vector Field Data
Dynamic Visualization Based on the Particle System

A Thesis Submitted to

University of Chinese Academy of Sciences

In partial fulfillment of the requirement

For the degree of

Master of Cartography and Geography Information System

By

Liu Hengxing

Thesis Supervisor: Professor Fan Xiangtao

Associate Supervisor: Associate Professor Liu Shuo

Institute of Remote Sensing and Digital Earth,

Chinese Academy of Sciences

June, 2017

中国科学院遥感与数字地球研究所

学位论文原创性声明和用户许可证说明

原创性声明

本人郑重声明： 所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。论文中凡引用他人已经发表或未发表的成果、资料、观点等，均已明确注明出处。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

本声明的法律责任由本人承担。

论文作者签名： 日期： 年 月 日

致 谢

本文研究得到中国科学院遥感与数字地球研究所数字地球理论与系统研究室的大力支持和说明，并受到中国科学院遥感与数字地球研究所“一三五”规划重大突破二项目“数字地球科学平台系统集成”课题(项目编号: Y3SG0300CX)、国家重点研发计划项目“地球资源环境动态监测技术”(项目编号: 2016YFB0501503)、海南省自然科学基金项目(项目编号: 20154171)和三亚市院地科技合作项目(项目编号: 2015YD21)的资助，在此致以衷心感谢!

刘恒星

2017年6月

摘要

随着数字地球技术的发展以及“下一代数字地球”概念的提出,数字地球将更加侧重于平台化构建以及对环境、健康、社会福利等重大问题的解决。党的十八大报告中提出,“提高海洋资源开发能力,坚决维护国家海洋权益,建设海洋强国”,基于数字地球系统对海洋环境模拟研究,深度挖掘海洋背后的规律,符合数字地球的发展趋势,意义重大。作为海洋环境模拟的重要组成部分,海洋矢量场数据的动态可视化方法的研究则必不可少。

目前,国内外在 Web 环境下对海洋矢量场数据动态可视化的研究主要存在三个问题亟待解决:一是可视化方式和效果,即选取合适的方法对海洋矢量场进行建模,使其能够生动、直观的表现矢量场的现状,而且能够动态连续地表现矢量场的基本变化趋势和规律。另外,目前的可视化方法大多数是从单一维度来查看,难以立体三维、多角度地对矢量场进行观察。二是可扩展的可视化引擎技术,目前很多 Web 的海洋环境数据可视化引擎不是一个完整的 WebGIS 引擎,不能够提供很多常见的 GIS 功能,导致后续进行可视化对象扩展时,难以自如对系统进行扩展。三是大规模可视化场景的渲染效率,即由于 WebGL 标准较新,很多 Web 三维场景加速方法不成熟,因此常见的提高渲染效率方法是尽量少的渲染场景内容,如可见性剔除,LOD(Level of Details)等,并未对整个可视化的流程采取多种方式协同的加速方法,故在效率提高方面并不明显。

针对上述三个问题,本文对 WebGL 技术下 SPH 实时流体模拟方法、SPH 原理简化的海洋矢量场动态可视化方法以及大规模三维可视化场景的渲染加速算法进行了研究与实现,取得的主要成果如下:

(1) 提出了一种 WebGL 技术下 SPH 实时流体模拟方法

本文提出了一个针对 WebGL 技术的基于多帧缓冲区的 SPH 实时流体模拟方法框架。针对 WebGL 技术的特点,在 GPU 中用多个纹理缓冲区来存储粒子位置以及相关的中间信息,从而减少了 CPU 和 GPU 之间的数据传输。同时,对邻域粒子的链表搜索算法改进,利用动态属性的方法对其进行优化,进而提升了邻域粒子的搜索效率。本文提出的算法在粒子数目增加至 10 万且进一步增加的情况下,

效率是经典的 Bayraktar 算法的 2 倍,能够满足在 Web 前端进行三维场景渲染的帧率要求。

(2) 提出了一种 SPH 原理简化的海洋矢量场数据动态可视化方法

在基于 WebGL 的 SPH 实时流体模拟方法的基础上,根据海洋矢量场数据的实时特点及其应用范围,对 SPH 方法的公式进行简化,并弱化了粒子系统中粒子所受外力的作用,将算法公式进行重新推导,最后给出整个算法实现的流程图。将复杂的 SPH 方法中核心思想运用于海洋矢量场数据的动态可视化,使得海洋矢量场的表达更加具有层次感与真实感。

(3) 实现了大规模矢量场数据可视化场景的渲染加速方法

本文综合研究和实现了多种渲染优化算法。采用 WebCL 并行计算、WebWorker 多线程和 V8 引擎分级内存回收算法来实时计算和更新粒子系统的各个粒子的状态,同时利用多帧缓冲区的 GPU 加速来实现场景的快速更新渲染。实验结果表明,在粒子数增加到 10 万以上,使用 WebCL 对 10 万以上粒子进行邻域粒子搜索效率能够提升 3 倍,而且使用 WebWorker 多线程使得在计算当前粒子状态信息时,速度至少提升 2 倍以上,在使用多帧缓冲区的 GPU 加速时,绘制并更新 20 万粒子的速度是传统 CPU 和 GPU 方法的 2 倍,综合使用以上方法使得整个场景的渲染帧率由 5 帧约提高了 15 帧。

关键词: 下一代数字地球, 数字地球科学平台, 海洋矢量场数据, 粒子系统, SPH

ABSTRACT

With the development of digital earth technology and the concept of "next generation digital earth", digital earth will focus more on platform construction and solving major problems such as environment, health and social welfare. The report of the 18th National Congress of the Communist Party of China puts forward that "improving the capacity of marine resources development and resolutely safeguarding the national maritime rights and interests, building a strong marine power", based on the digital earth system to simulate the marine environment, it deeply excavates the laws behind the ocean and conformed to the development trend of digital earth significantly. As an important part of the marine environment simulation, the study on the dynamic visualization method of marine vector field data is indispensable.

At present, there are three problems in the research of dynamic visualization for marine vector field data at domestic and abroad: one is visualization and effect, that is, we choose the appropriate method to model the ocean vector field so that it can be vivid and intuitive. And the status of vector field can dynamically and continuously show the basic trend and law of vector field. In addition, it is difficult to observe the vector field with three-dimensional, multi-angle. Second, the scalable visualization engine, many Web's marine environment data visualization engines are not a complete WebGIS engine, and it is difficult to freely expand the system. Third, the large-scale visual scene rendering efficiency, because the WebGL standard is newer, many Web three-dimensional scene acceleration methods are not mature, so the common efficiency improvement method is as little as possible rendering scene content, so the efficiency of the increase is not large.

In the view of above three problems, this paper introduces the SPH real-time fluid simulation method, SPH principle simplified marine vector field visualization method, the building of Web digital earth system based on Cesium and the rendering acceleration algorithm of large-scale 3D visualization scene Research and implementation, and the main results achieved are as follows:

(1) A method of SPH real-time fluid simulation based on WebGL technology is proposed. This paper presents a SPH real-time fluid simulation method framework based on multi-frame buffer for WebGL technology. For the characteristics of WebGL

technology, it should use the GPU with multiple texture buffer to store the particle location and the relevant intermediate information. At the same time, the search algorithm of domaining particles is optimized by dynamic attribute method, and the search efficiency of neighborhood particles is improved. The efficiency of this algorithm is twice as high as GPU-based Bayraktar algorithm, and it can meet the frame rate requirement of 3D scene rendering at Web front end. when the number of particles increases to 50k, it will further increase.

(2) A dynamic visualization method of marine vector field data with SPH principle is proposed. Based on the WebGL-based SPH real-time fluid simulation method, and according to the real-time characteristics of marine vector field data and its application range, the formula of SPH method is simplified. The algorithm formula Re-derivation, and finally gives the entire algorithm to achieve the flow chart. The core of complex SPH method is applied to the dynamic visualization of the ocean vector field, which makes the expression of ocean vector field more layered and realistic.

(3) a large-scale vector field data visualization scene rendering acceleration method is achieved. This paper comprehensively studies and implements a variety of rendering optimization algorithms. WebWorker multithreading and V8 engine hierarchical memory reclamation algorithms are used to calculate and update the state of each particle in the particle system in real time, and the GPU is accelerated by multi-frame buffer to realize the rapid update rendering of the scene. The experimental results show that the number of particles can be increased by more than 100,000, and WebCL can be used to enhance the efficiency of neighborhood particle search by 3 times, and the WebWorker multithreading is used to increase the speed by at least 2 times when calculating the current particle state information. when multi-frame buffer GPU acceleration is used, the speed of drawing and updating 200,000 particles is 2 times than the traditional CPU and GPU method, and the integrated above methods makes the rendering frame rate of entire scene from 5 frames increased by about 15 frames.

Key Words: Next-generation Digital Earth, Digital Earth Science Platform(DESP), Ocean Vector Field Data, Particle System, Smoothed Particle Hydrodynamics(SPH)

目 录

摘 要	I
ABSTRACT	III
目 录	V
1. 绪论.....	1
1.1 选题背景.....	1
1.2 选题意义.....	2
1.3 国内外研究现状.....	5
1.3.1 海洋矢量场动态可视化研究现状.....	5
1.3.2 SPH 流体实时模拟研究现状	6
1.3.3 Web 数字地球系统研究现状	8
1.4 论文研究内容与创新点.....	11
1.4.1 研究内容	11
1.4.2 创新点	13
1.4.3 论文组织结构	14
2. 基于粒子系统的海洋矢量场数据动态可视化方法	15
2.1 海洋矢量场数据解析与预处理.....	15
2.1.1 海洋环境数据概述	15
2.1.2 NetCDF 概述和预处理	16
2.2 海洋矢量场数据可视化的主要方法.....	20
2.2.1 点图标法	20
2.2.2 向量线法	21
2.2.3 特征法	21
2.2.4 粒子系统法	21
2.3 基于 WebGL 的 SPH 实时流体模拟方法.....	22
2.3.1 光滑粒子流体动力学 (Smoothed Particle HydroDynamics)	22
2.3.2 基于 WebGL 的实时流体模拟方法原理	24
2.3.3 方法实现过程	25

2.3.4 实验结果与对比分析	29
2.4 SPH 原理简化的海洋矢量场数据动态可视化方法	30
2.5 本章小结	35
3. 大规模矢量场数据可视化场景的渲染加速方法	37
3.1 WebGL 并行计算	37
3.1.1 WebGL 实现	38
3.2 WebWorker 多线程	40
3.3 基于 V8 引擎的分级内存回收算法	42
3.4 多帧缓冲区的 GPU 加速算法	45
3.5 本章小结	46
4. 海洋矢量场数据动态可视化的设计与实现	47
4.1 基于 Cesium 的 Web 数字地球原型系统构建方法与实现	47
4.1.1 自定义 shader 的多通道渲染	47
4.1.2 基于 Primitives 的复杂几何体渲染方法	50
4.1.3 Requirejs 模块化代码组织方式	53
4.1.4 基于 Cesium 的 Web 数字地球原型系统实现	54
4.2 基于粒子系统的海洋矢量场数据动态可视化	57
4.2.1 基于粒子系统的海洋矢量场数据动态可视化设计	57
4.2.2 基于粒子系统的海洋矢量场数据动态可视化实现与分析	59
4.3 本章小结	64
5. 结论与展望	65
5.1 结 论	65
5.2 展 望	66
参考文献	67
致 谢	71
作者简历及攻读学位期间发表的学术论文与研究成果	72

1. 绪论

1.1 选题背景

中共十八大报告提出,提高海洋资源开发能力,发展海洋经济,保护海洋生态环境,坚决维护国家海洋权益,建设海洋强国。21 世纪是海洋的世纪,海洋事关国家安全和长远发展,世界主要海洋国家将海洋权益视为核心利益所在,积极推行新一轮海洋经济政策和战略调整。中国作为世界第二大经济体,已发展成为高度依赖海洋的外向型经济,对海洋资源、空间的依赖程度大幅提高。中国是一个拥有 300 多万平方公里海域、1.8 万公里海岸线的大国,建设海洋强国、维护海洋权益是发展之要、民生之需,也是中国海洋权益维护和拓展的题中之意。

2012 年, M. Goodchild 和郭华东院士等共同撰写发表在《美国国家科学院院刊》(PNAS) 的论文《Next-generation Digital Earth》(Goodchild et al., 2012), 回顾了数字地球这门年轻的学科走过十余年历程取得的成就,并展望了数字地球科学的前景。1998 年, 戈尔首次提出的“数字地球”理念(Core, 1998), 第一代的数字地球实现地图的基本操作、具备带有高程属性的地球数据,并采用 API、KML 等技术手段满足用户需求,产生许多实际应用。戈尔当年提出的设想,目前大部分已经现实,不过仍存在一些问题。比如数据精度不尽如人意,还存在不确定性、不可复制性等;不能有效利用历史数据对未来进行预测;没有建成对地球相关信息的存储、检索和共享环境;信息的表达手段单一,如温度场、生物多样性、社会和经济的分布等缺乏有效的表达手段。

另外,在云计算、物联网、大数据等新技术的推动下,在全球变化、区域可持续发展需求的牵引下,国际数字地球学会(ISDE)的科学家团体提出了新一代数字地球的概念(Craglia et al., 2012)。空间数据基础设施、分布式数据和处理及其互操作与集成、地学浏览器、对地观测网络、高性能计算等方面的发展为实现下一代数位地球提供了驱动力。数字地球的发展需要融合这些进展,借鉴 Web2.0、社会网络(Social Network)、无处不在的网络社会(Ubiquitous Network Society)

和 VGI (Volunteered Geographic Information) 理论和技术, 克服 SDI 和 GEOSS 在数据描述、组织管理、搜索、地学处理服务、动态数据分析和建模、语义显示描述、模型互操作以及共享策略等方面的不足, 进而呈现给人们一个功能和信息更加丰富、易用, 能够更好服务我们的全新的数字地球。

在地学领域中, 矢量场描述了许多非常重要而且常见的物理现象。矢量场可视化是理解矢量场运动变化规律的有力工具, 在科学计算和工程分析中发挥着重要作用, 已被广泛应用于地质勘探、天文气象、航空航天等领域, 具有非常重要的实际意义和研究价值。近年来, 随着计算流体力学的快速发展, 迫切需要一种能够有效说明大量数据集中所蕴含规律的技术, 特别是在工程计算、地学领域, 矢量场的研究者需要耗时更少, 性能更好的可视化方法 (Wu et al., 2004)。

目前能够描述矢量场的方法有点图示法、矢量线 (迹线、流线)、矢量面 (流面、流管) 等。点图示法和矢量线 (面) 法, 有一个共同的缺点: 采样频率太小会使可视化结果忽略矢量场的一些重要细节, 而过多的采样又容易导致图像混乱, 不易于观察流场内部细节。目前的矢量场可视化研究在以上问题的研究中, 还没有达到令人满意的效果, 因此本研究采用改进的 SPH 算法, 根据得到数据构建粒子系统, 并以海洋风场为例, 实现海洋风场在 Web 数字地球系统的动态可视化。

1.2 选题意义

基于三维虚拟地球的数据可视化具有直观、形象的优点, 在此场景下进行多尺度、大范围数据的三维展示具有先天优势, 可为数据展示和变化规律分析提供更为直观的可交互可视化平台。按照地学数据的类型, 数据的可视化主要包括标量场可视化、矢量场可视化以及多种类型数据的迭加可视化, 在三维虚拟地球中进行地学数据的可视化展示, 不仅可直观地揭示多维地学数据间的各种关系及内在规律, 而且可通过灵活多样的显示手段实现地学数据的多维、动态展示。

近些年来, 三维球体软件由于其丰富的空间信息展示能力和强大的三维空间分析功能, 已在众多领域取得成功应用, 得到了众多行业用户的广泛认同, 尤其在城市规划、数字城市、智能交通、虚拟旅游、军事模拟等领域备受亲睐。随

着“数字地球”、“智能地球”建设的深入及其理念的广泛接受,国内外涌现出了一大批优秀的三维 GIS 软件或模块,如国外谷歌的 Google Earth、NASA 的 World Wind (World Wind Resource Center, 2004)、ESRI 公司的 ArcScene (ESRI, 2015)、ArcGlobe 和 ArcGIS Explorer (ESRI, 2015)、微软公司的 Virtual Earth 3D (Microsoft Inc, 2015)、Skyline 公司的 Skyline Globe, 国内的国遥新天地信息技术有限公司的 EV-Globe (EV-Globe, 2014)、中国科学院遥感与数字地球研究所的数字地球科学平台 (DESP) (杨林平, 2013) 等多种三维软件。但是, 由于 C/S 架构的限制, 当前三维球体可视化系统仍存在着维护成本高、系统版本更新难、维护任务量大等问题。B/S 架构模式是一种 Web 应用开发的新模式, 用户通过浏览器与服务端交互来使用系统功能, 而业务逻辑则全部放在了远程的服务器中, 从而降低了开发、部署和维护的成本, 这种架构模式也成为目前很多系统开发的首选模式。

同时, 2011 年上半年发布的 WebGL 是一种在浏览器中的 3D 绘图标准 (Khronos Group, 2011), 它允许浏览器中的 Canvas 标签直接使用 GPU 进行加速 3D 渲染, 从而使在网页上实现高效率的 3D 图形程序成为了可能, 这使传统的可视化程序从 C/S 转向 B/S 成为了可能。这些新发布的 Web 标准和技术使得在浏览器端构建功能丰富的三维应用成为现实。随着技术的不断完善, 不仅是传统 PC 机, 像手机、Pad 等支持浏览器的移动设备也可以运行 Web 数字地球系统, 使普通的用户和科研人员得到便利和良好的用户体验。

目前, 目前的矢量场可视化多采用点图示法和矢量线法, 点图示法虽然简单, 易实现, 但存在很多缺陷。采样过密, 容易导致图像杂乱无章, 显示太少, 又不能准确把握矢量场的变化情况, 而且该方法难以揭示出数据的内在连续性如图 1-1 和图 1-2。矢量线较点图示法有了很大改进, 他能表示场的连续性, 绘制效果较好, 通过初始点的位置适当选取, 可以显示矢量场重要的特征和细节。但两者均不能很好的表示矢量场的量的大小和直观的显示矢量场的运动情况, 这不符合矢量场在实际中所代表的情况 (葛成, 2011)。



图 1-1 点图示法和矢量线法表示风场（王想红，2013）

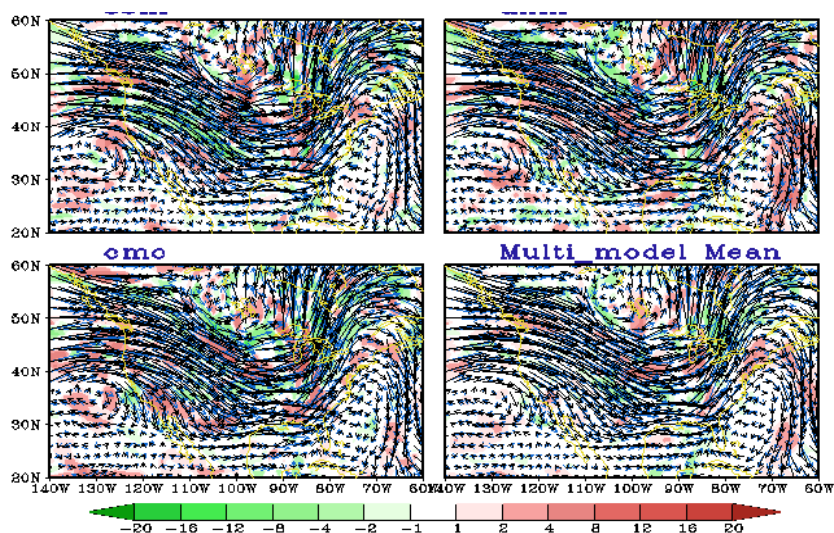


图 1-2 NOAA 利用箭头来表示矢量场 (<http://nomads.ncep.noaa.gov/>)

本研究以中科院遥感与数字地球研究所自主研发的数字地球科学平台为依托，以开源 Web 数字地球框架 Cesium 为平台，利用 WebGL 技术，建立新的 SPH 流体模拟框架，同时根据海洋矢量场数据的特点以及海洋矢量场的特征，简化了 SPH 方法，并将其用在海洋矢量场可视化实现上，以海面风场为例，在 Web 数字地球系统上实现海洋风场的动态可视化，这有效的解决了桌面端数字地球系统难以跨平台，维护成本高等问题，以及海洋矢量场在 Web 数字地球系统上的直观动态可视化效果。

1.3 国内外研究现状

1.3.1 海洋矢量场动态可视化研究现状

目前,常用的向量场资料可视化方法主要有四大类,分别是基于几何形状的向量场映射方法、基于颜色、光学特性的向量场映像方法、基于纹理的向量场映像方法以及特征可视化方法(石鑫祥,2011)。这些方法优缺点明确,已在不同的应用领域得到了广泛应用,取得了不错的效果。

基于几何形状的向量场映射方法常用的有点图示法、向量线法、向量面法等。该方法具有信息准确、直观等优点,但很难全面、连续地反应向量场,只适应于较简单的向量场可视化,是目前向量场可视化中用的最多的方法。Max 和 Crawfis 在 visualization 上提出用头发作图示附着在向量场中的面上来表示向量场中表面附近的向量方向,该方法在面上的采样点处画上一根根短线,然后让这些线沿向量的方向弯曲,很像一般的头发。韩冬(2005)对大尺度海洋风场数据进行处理,在 MAGIS 系统中应用 MFC 实现了点图法的风场可视化。宋丽丽(2013)采用向量箭头表示方向,颜色表示值对海洋向量场资料进行了可视化。Kenwright (1992)等人改进了传统的流线生成方法,提出了基于流函数的向量线构造方法。李文庆(2011)将海洋表面数据的可视化与海洋水面的模拟进行一体融合,实现了海洋表面场数据的点、线、面、体等形式展示。王想红(2013)基于改进的三维箭头点图示法和向量线法实现了风场数据的动态可视化。葛成(2011)研究了在地学机理与过程中所产生的三维向量场数据的三维可视化方法,并实现了向量场的点图示法、向量线法、基于自适应采样密度的向量带和向量面法,取得了良好的效果。但是,由于这类方法是建立在离散采样的基础上,当采样密集时,会产生视觉上的混乱;同时,采样稀疏又会丢失向量场中重要的特征和细节,因此,它很难全面地、连续地反映向量场。

基于颜色、光学特性的向量场映像方法中颜色映射常用于标量场的标量值与颜色的映射,如果表示向量场,则需对颜色映射的方法予以扩展,Sakas(1993)成功地用体绘制方法动态地模拟了湍流气体的运动,他用频谱合成技术生成了随机的、随时间变化的密度场。田玲(2007)提出变换纹理序列的思想对基于二维纹理映像的体绘制算法进行改进,并且通过计算体数据包围盒与垂直与视线的平面的切片多程序设计实现了三维纹理映像体绘制算法,实现了较好的绘制效果。粒子方法是此类方法中另一个应用较多的方法,Reeves(1983)提出了一种对不规则模糊物体的建模方法,即粒子方法。由于粒子属性映像能较好地反映流场的变化和状态,近些年来,粒子方法开始被引入向量场可视化领域。此后,Reeves

(1985)对粒子理论作了进一步扩展,将粒子系统中的粒子由发光体推广到可反射、折射外部光的实体。以此为理论基础,人们常将粒子系统用于更加复杂物体的造型,目前已经能较好地模拟出模糊物体(如树、烟雾等)的动态性和随机性。Sims(1990)提出了一种更一般更灵活的绘制粒子的方法,每个粒子都有一个头和一个尾,头和尾都有自己的位置、半径、颜色、不透明度等属性。宋刚(2007)在粒子系统的基础上,通过对火焰运动的分析,提出了一种基于粒子系统的风场模拟算法,并基于 OpenGL 实现了不同的火焰模拟效果。肖凯涛(2015)等提出一种将计算流体力学技术的风场结果与粒子系统相结合的方法,并在模拟平台 Unity 3D 中实现了风场在虚拟环境中的可视化。于海亮(2008)等基于 Lagrange 框架下的粒子追踪方法数值模拟海上溢油的三位运输模式,并与卫星遥感监测数据比较,得到较好的模拟效果。李骞(2005)等采用粒子系统的方法来可视化风场,同时将风场涡旋度的强弱映射成粒子本身的旋转状态,并验证实验结果。用粒子来显示向量场,灵活、方便,是较为理想的一种表达方法。

基于纹理的向量场映像方法包括点噪声法、线积分卷积法和可变形参数域卷积法,特征可视化方法包括向量场拓扑结构分析法和流场中的特征结构的可视化,这两类方法理论较为抽象,本身相对复杂,需要较高的程序设计水平,目前很少有相关学者采用这两类方法(1999)。综合以上分析来看,在实现向量场动态可视化效果和实现难易程度的综合考虑下,粒子方法是一个较为理想的方法,在向量场可视化中可将粒子的某一具体性质与向量场中的向量联系起来,能够得到较为理想的可视化效果。粒子系统方法可以说是一种基于物理模型来解决问题的方法,它的核心不在于如何显示图形,而是在于对某个物理模型的理解和分析,即对微小物质模型的规则提取,粒子的运动规律可以很简单也可以很复杂,这取决于模拟的物理模型的复杂程度。采用动画形式的粒子运动能真实、有效地可视化流场变化。固定显示刷新时间和时间步长,粒子不同时间所处的空间位置直接表示了风场中的速度变化。但目前关于粒子系统方法的研究在这两点有一定的不足,适当选取物理模型,并针对粒子系统本身的追踪算法进行改进,使其更符合实际情况,达到直观逼真的向量场可视化效果。

1.3.2 SPH 流体实时模拟研究现状

光滑粒子流体动力学方法(Smoothed Particle HydroDynamics,以下简称 SPH)是近 20 多年来逐步发展起来的一种无网格方法,1977 年 Lucy(1977)提出将连续的流体用相互作用的质点来描述,通过求解各个质点的动力学方程和跟踪每个质点的运动轨迹,最后求得整个系统的运动力学行为。

光滑搜索函数在 SPH 方法中起着重要的作用,因为它决定了函数表达式的

精度和计算效率,在流体模拟过程中,都需要对以 h 为半径的范围内的粒子的搜索占用了主要的时间(杨旭波,2014)。当粒子数量较多时,搜索的效率对最后的模拟效果更是为产生较大的影响。

针对整个 SPH 流体模拟过程中的难点,国内外的相关研究人员都做了大量的工作,以期在效率和效果上达到较好的用户体验。Muller(2003)提出了一种基于平滑粒子动力学的粒子来模拟流体的运动,并直接从 Navier-Stokes 方程中导出力密度字段和提出了计算粘度和速度的光滑核函数。林昊(2011)利用一系列 SPH 流体模拟中涉及的常用算法,结合浅水坡方程对水流的震荡和流水的塌方溃坝进行模拟,并通过 OpenGL 进行了实现。陈英建(2012)利用基本的 SPH 流体模拟理论,并针对相邻粒子搜索的熟悉算法进行改进,提出多维树搜索算法,最后构建了管道流体模拟的实验平台实现了管道流体的流动和流体形成的漩涡和喷射等效果。赵相坤(2011)等提出了一种在 GPU 上实现的基于网格的邻居快速查找算法,成倍地提高了流体模拟的速度。Harada(2007)等提出了一种基于 SPH 的流体模拟系统,该系统使用桶纹理来表示 3D 的网格结构并实现了一种高效的邻居查找算法,但每一个网格内最多只能有 4 个粒子。Bayraktar(2009)等提出了一种在 GPU 上实现的基于网格的邻居查找算法,复杂的纹理产生过程使得该算法虽然可在 GPU 上实现,但是速度低于同类基于排序网格的方法。唐勇(2013)等对 SPH 中相邻粒子搜索算法进行了改进,提出一种基于 GPU 的多维树搜索算法,使流体模拟达到了更好的实时性效果。郭秋雷(2011)等借助基于 GPU 的空间划分 PSS 和设计流体表面信息的重建算法,并进一步实现了基于索引的优化,最后提出一套基于 GPU 的 SPH 流体模拟加速框架。周梟楠(2015)等针对 DEM 资料特点,对 SPH 的边界条件处理方法进行修改,利用 OpenGL 的显示能力,在两种不同类型的地形上对水体流动进行实时计算和模拟。Xia(2016)等利用 GPU 的并行计算能力,并设计一个基于 GPU 的 SPH 模型用来求解可变光滑长度 SWEs 问题,最后对两个真实世界的溃坝情况进行模拟,证明了所提出的基于 GPU 加速的 SPH-SWE 模型的高性能表现效果。

目前常用的 SPH 邻域粒子搜索算法包括三种,分别是全配对搜索法,链表搜索法和树形搜索法(何戡,2010)。针对以上三种搜索算法,其中最为直接而简单的是全配对搜索法,针对给定粒子,需要对整个计算区域内的所有粒子进行全扫描,该算法时间复杂度为 $O(n^2)$,由于其时间复杂度太高,在实际模拟中较少使用此搜索算法。树形搜索法非常适合求解可变光滑长度的情况,每一步计算均需要通过粒子的位置来构造有序树,其时间复杂度为 $O(n \log n)$ 。树形搜索适合于求解可变光滑半径和粒子数较多情况,另外在树的构造和算法理解上相对复杂,本文暂不采用此算法。而链表搜索对作用域内的粒子进行区域划分,将粒子

位置和区域索引存储在 Hash 表内, 然后对 Hash 表进行遍历即可, 时间复杂度为 $O(n)$ (靳超, 2015)。因其算法理解上较为简单且在程序上容易构造, 本文采用此算法, 并对排序好的粒子和区域网格的 Hash 表中不含粒子的网格 Hash 值进行修正。另外, 本文在 WebGL 中进行流体模拟时, 利用着色器将流体模拟的中间过程写入 GPU 中执行, 使得整个流体模拟过程只需完成初始化, 其余计算便完全在 GPU 中执行, 并且将多次计算结果写入同一帧缓冲区, 最后一次性交给主缓冲区进行渲染, 实现了流体模拟在 Web 浏览器中的快速渲染。

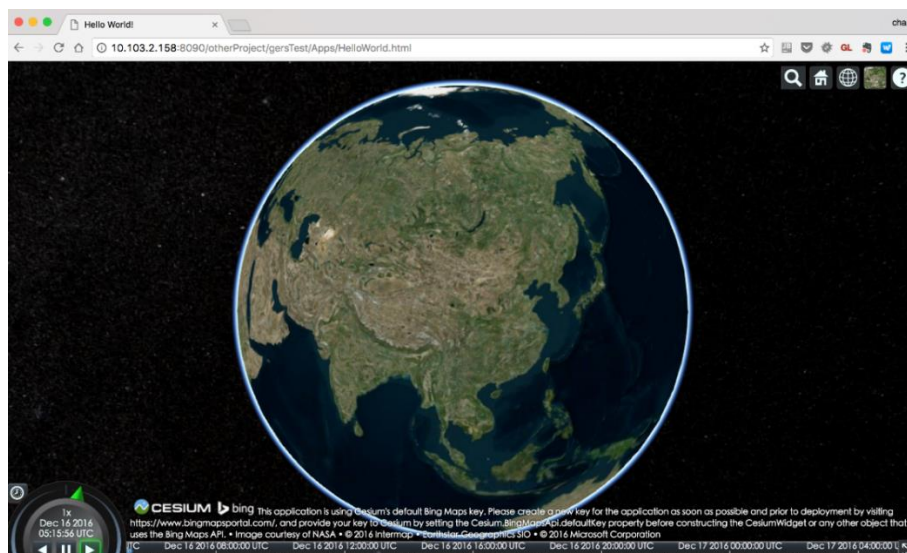
1.3.3 Web 数字地球系统研究现状

由于 WebGL 的强大功能和 HTML5 标准的最终定稿, 这都大大增强了 Web 应用程序的三维渲染功能, 使得越来越多的研究机构和开发人员从传统的桌面三维 GIS 应用程序转向了 3D WebGIS, 并产生了一系列效果惊艳的产品。

1) Cesium

Cesium 是一套用来渲染 3D 地球, 2D 区域地图, 和多种 GIS 要素的 JavaScript 库。不需要安装任何插件就能在支持最新 HTML5 标准的浏览器上运行。Cesium 支持 WebGL 硬件加速, 非常适合动态数据在 GIS 图层上的展示, 是一个跨平台, 开源, 非常有前景的 WebGIS 表现层库, 并且它是在开源的 Apache 2.0 许可下, 可免费用于商业和非商业用途 (Analytical Graphics Inc, 2015)。

Cesium 源自 Analytical Graphics, Inc. (AGI) 公司为他们客户开发的一个虚拟地球项目, 后来将 Cesium 项目贡献给开源社区并一直活跃开发中, 并且基于 Cesium 也产生了很多功能较强的应用, 是当前最有潜力的一个三维 Web 数字地球系统。(AGI 开发的 STK Terrain Server 是一种针对 Cesium 应用的流式高分辨率服务器) 另一个公司级的贡献者是 NICTA (National ICT Australia) 澳大利亚最大的信息与通讯技术中心, NICTA 的员工们贡献了瓦片地图服务、地形处理等 Cesium 的核心功能。并一直使用 Cesium 开发环境监测分析、交通基础设施建模仿真优化等领域的应用。

图 1-3 cesium 渲染效果图 (<http://cesiumjs.org/>)

2) WebGL Earth

2011 年, Petr Sloup 发表了署名为“WebGL Earth”的论文(Sloup, 2011), 论文实现的关键部分为纹理管理和渲染方法。论文分析了现有解决方案, 评估在指定环境中的可用性, 并提出了新的更合适的结构和算法(TileBuffer with binary lookup (引入元数据描述, 取代 Virtual Texturing 的查询表, 以解决其不易扩展的问题)和 ClipStack (限定启动状态的 ClipLevels 数量, 以减少系统计算负荷)), 并且最终结果是一个在浏览器中实时呈现的三维虚拟地球, 就是 WebGL Earth。同年, 由 Petr Sloup 负责的开源爱好者小组(WebGL Earth Developer Group)开发的 WebGL Earth 上线, 它实现了多种 WebGIS 数据源(包括 OpenStreetMaps、MapQuest、Bing Maps 等)的高精度纹理、DEM 加载和类似 Google Earth 的漫游方式(Klokan, 2015)。

图 1-4 WebGL Earth 效果示意图 (<http://www.webglearth.com/>)

3) OpenWebGlobe

OpenWebGlobe 是一个高性能三维引擎, 它还包括在高度并行和可伸缩的计

算环境下处理大量的地理空间数据的软件。WebGL 版本在浏览器中运行不需要插件。OpenWebGlobe 支持多种数据类别如图像数据、高程数据，感兴趣的点，向量数据和三维模型。数据在互联网上以流的形式传输之前，这个数据必须经过预处理。这种预处理通常包括转换地方到全球参考系统，创建金字塔层或 LOD，数据切片处理 (Martin, 2015)。因为数据集通常很大，包含兆兆字节，先进的核外渲染机制中详细级别的方法可用于可视化。另外，在开放系统的同时，还提供了 SDK (Software Development Kit)，主要由 Viewer part 和 Processing Tools 两部分组成，Viewer part 负责场景组织与生成和 Web 可视化，Processing Tools 由一系列工具组成，负责对地形、影像、点云数据等进行预处理 (切片、重采样等)，供 Viewer Part 使用 (Loesch et al., 2012)。



图 1-5 OpenWebGlobe 效果示意图 (<http://www.openwebglobe.org/>)

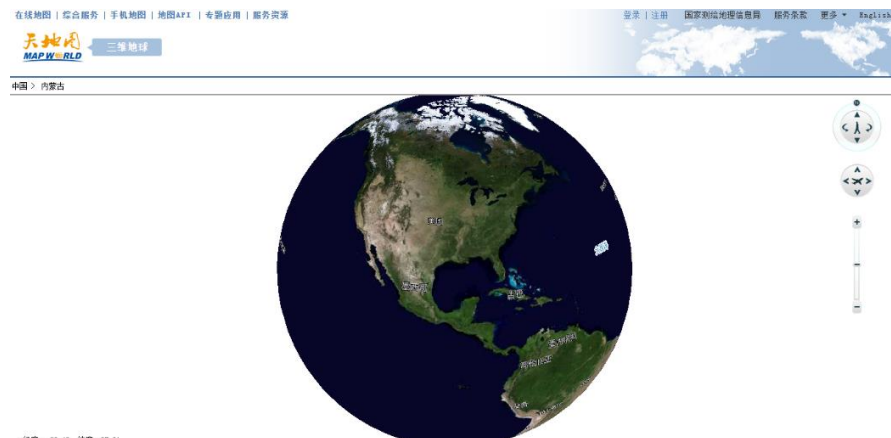
4) 其他类似的产品

2011 年，Google 启用 MapGL，在浏览器中实现 3D 版地图，系统性能大大提升，不仅地图加载速度提高了，而且进入街景模式更快且平滑。国内的天地图和百度地图也推出了类似的三维地球，具有三维地球的基本架构，如空间参考系统、球面格网系统和瓦片层次系统等，并且可以进行一些基本的图层管理操作，但在功能扩展上相对较差。

综合来看，相较于同类产品，Cesium 是目前发展较好的一个 web 数字地球框架，该框架依托于 AGI 公司，每月更新一次版本，并且其论坛相对活跃很多。Cesium 可以兼容各种地图服务和较多的向量数据格式，另外其具有较强的图形绘制能力和自由度较高的开放界面。本研究采用 Cesium 来搭建 Web 数字地球系统，利用其灵活的可扩展性，实现矢量场动态可视化效果 (Bernd et al., 2014)。



图 1-6 Google Map 示意图

图 1-7 天地图三维地球示意图 (<http://www.tianditu.cn/>)

1.4 论文研究内容与创新点

1.4.1 研究内容

在基于粒子系统的 Web 数字地球上进行大规模的海洋矢量场可视化研究，本文的研究内容主要分为以下三个方面：

1. 基于粒子系统的海洋矢量场数据动态可视化方法

基于海洋矢量场数据的特点及其应用范围，本文在基于粒子系统对其可视化时，首先，对 WebGL 环境下的 SPH 流体模拟方法研究，提出基于 WebGL 的多帧缓冲区的 SPH 流体粒子初始化更新和渲染过程的方法，该方法首先对传统的邻域粒子链表搜索算法进行改进，然后在频繁的粒子位置与速度更新过程中，提出基于多帧缓冲区的中间数据存储和传输方法。最后主要介绍了基于 SPH 进行海洋矢量场数据动态可视化的算法的难点和关键点，并给出其中改进后的公式推导，最后给出整个算法实现的流程图。

2. 大规模矢量场数据可视化场景的渲染加速方法

在 WebGL 中进行 SPH 的流体模拟，需要实时计算和更新粒子系统的各个粒

子的状态，这对 JavaScript 代码和硬件环境提出了很高的要求，需要我们在进行动态可视化的同时利用多项技术和渲染优化方法来达到最好的渲染效果。因此，本文综合研究和实现多种渲染优化算法，如 WebCL，WebWorker 多线程、V8 引擎分级内存回收算法和多帧缓冲区的 GPU 加速算法，使海洋矢量场动态可视化可以在 Web 数字地球系统上流畅的运行和交互，以满足大规模海洋矢量场数据模拟的逼真、高效、科学需求。

3. 基于 Cesium 的 Web 数字地球系统构建方法与实现

大规模海洋矢量场的动态可视化需要在大尺度的范围上展示才能查看其总体的运行趋势，基于此，本文基于 Cesium 引擎构建 Web 数字地球系统，并且在自定义 shader 的多通道渲染，基于 Primitives 的复杂几何体渲染方法和 Requirejs 模块化代码组织方式三种方式下，研究如何根据个性化的需求构造一个面向特殊领域的 Web 数字地球系统。具体包括，研究其底层 WebGL 的管道流水线的渲染原理，包括顶点着色器、片元着色器以及纹理映像等内容。同时，在 RequireJs 规范下，对 Cesium 底层架构进行扩展和自定义，进而实现对 Web 数字地球系统构建研究。

论文的总体研究思路如图 1-8 所示。

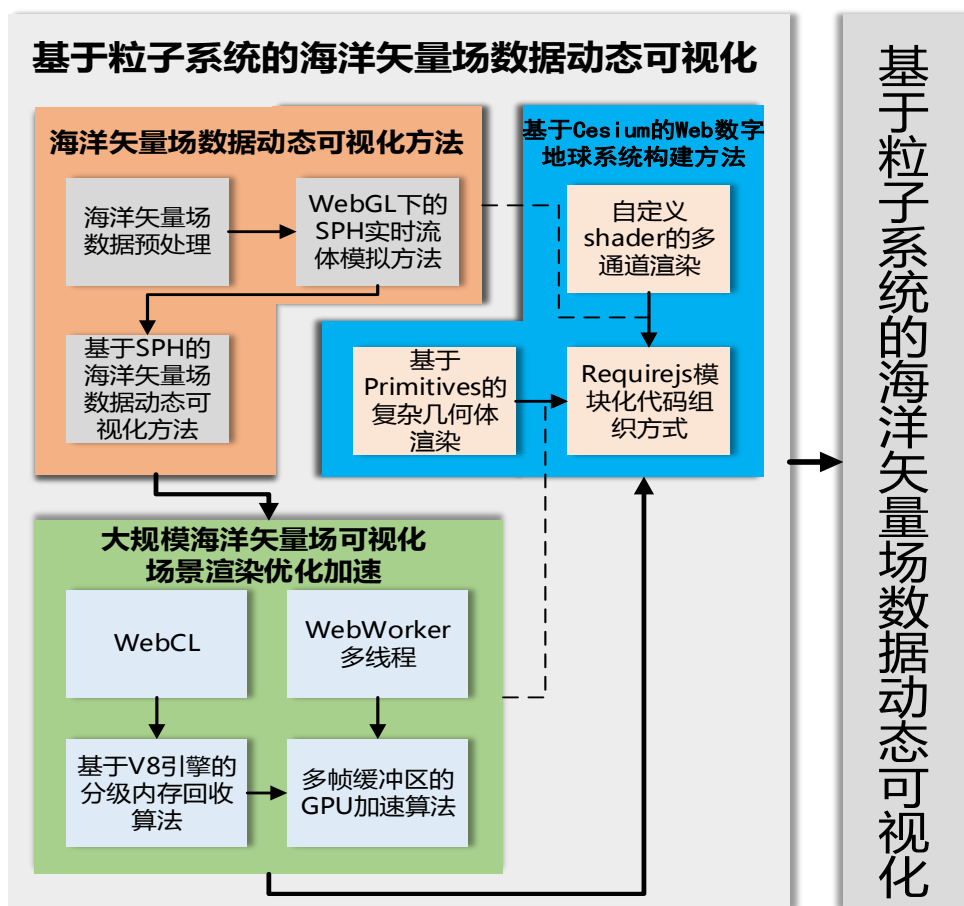


图 1-8 论文研究总体思路示意图

1.4.2 创新点

针对国内外关于海洋矢量场数据动态可视化方法的研究与实现过程中存在的问题进行了以下两点创新：

1. 提出了一种新的基于 WebGL 的 SPH 实时流体模拟方法

计算机流体动力学中进行实时流体模拟时，常采用刚兴起的无网格方法 SPH 来进行，而且在 OpenGL 下进行了较多的应用。WebGL 标准的提出，使得在 Web 环境下无插件进行大规模的场景渲染成为可能。本文提出了一个针对 WebGL 技术的基于多帧缓冲区的 SPH 实时流体模拟具体框架，分为初始的初始化，更新和渲染的过程，对原有 SPH 整个算法的具体实现，针对 WebGL 技术的特点，在 GPU 中用多个纹理缓冲区来存储粒子位置以及相关的中间信息，从而减少了 CPU 和 GPU 之间的数据传输。同时，对其中的邻域粒子的链表搜索算法，利用动态属性的方法对其进行优化，进而提升了邻域粒子的搜索效率。

2. 提出了一种 SPH 原理简化的海洋矢量场数据动态可视化方法

本文在基于 WebGL 的 SPH 实时流体模拟方法的基础上，根据海洋矢量场数

据的实时和其本身场的特点,对 SPH 方法的公式进行简化,并弱化了粒子系统中粒子所受外力的作用,将算法公式进行重新推导,最后结合 WebCL、WebWorker 多线程,V8 引擎的分级内存回收等算法对大规模的海洋矢量场数据模拟场景进行渲染加速,使得整个海洋矢量场数据的动态模拟能够在 Web 环境下流畅的运行。

1.4.3 论文组织结构

根据本文的研究内容,本文通过六个章节进行描述,具体章节安排如下:

第一章,绪论。阐述了本论文的研究背景、研究意义、国内外关于海洋矢量场数据动态可视化、SPH 流体实时模拟和 Web 数字地球系统发展现状以及本文的研究内容与创新点。

第二章,基于粒子系统的海洋矢量场数据动态可视化方法。在基于粒子系统的海洋矢量场数据动态可视化方法中阐述了海洋环境数据、海洋矢量场数据,NefCDF 格式以及 NefCDF 的预处理方法,同时对海洋矢量场可视化方法进行对比分析。在基于 WebGL 的 SPH 实时流体模拟方法部分介绍了 SPH 方法在 Web 环境下应用的一个框架。最后主要介绍了利用 SPH 进行海洋矢量场数据动态可视化的难点和具体解决方法。

第三章,大规模矢量场数据可视化场景的渲染加速方法。阐述了对大规模海洋矢量场数据动态可视化的四种渲染加速方法,其中 WebCL 和 WebWorker 多线程主要用于对算法中逻辑计算型任务进行分解和快速计算,从而减少 Web 前端在这个阶段的时间损耗。同时,基于 V8 引擎的分级内存回收算法和多帧缓冲区的 GPU 加速算法分别用于减少渲染过程中内存的浪费和减少 CPU 与 GPU 数据频繁的交流,最后均致力于提高整体 WebGL 三维应用的运行和渲染效率。

第四章,海洋矢量场数据动态可视化的设计与实现。阐述了基于 Cesium 构建 Web 数字地球系统的三种方法,三种方法提供了非常灵活的方式,根据用户需求,方便用户构建个性化的 Web 数字地球系统。在基于粒子系统的海洋矢量场数据动态可视化上,本文对其具体实现进行设计,并结合多种大规模场景渲染加速的方法,对整体的场景进行了实现,并给出实验结果的展示。

第五章,结论与展望。对本论文的相关工作进行全面总结,对现存的问题进行分析和讨论,并提出今后需要进一步深入研究的工作。

2. 基于粒子系统的海洋矢量场数据动态可视化方法

“可视化”(Visualization)是通过图形图像等可以看见并认证的手段,来形象表现科学数据的架构和内涵。使用图形来表示信息的主要优点是赋予信息某种形态,其目的是辅助我们分析信息及信息之间的关系,减少理解和认知它们所需的努力。与标量场数据相比,矢量场数据最大的不同点在于每个物理量不仅具有大小,而且具有方向,因此,矢量场的可视化方法与标量场有所不同且更为复杂。同时,矢量场数据预处理主要围绕矢量场数据点之间拓扑结构复杂和数据量大的特点进行的,这都为海洋矢量场数据的可视化提出了很多难题。

2.1 海洋矢量场数据解析与预处理

海洋环境是一个多耦合,多要素,时变性很强的复杂巨系统,海洋环境数据是认识海洋现象的必要途径,海洋矢量场数据是海洋环境数据的一部分,典型的矢量场数据如海洋风场和海洋洋流场,它们反映了海洋表层一些具有方向性的海洋环境数据,而这些动态数据又会影响诸如温度、盐度、密度、潮汐和风暴潮等。

2.1.1 海洋环境数据概述

海洋数据的可视化方法主要由数据本身的类型和特点决定。其中狭义的数据类型指数据自身的类型,广义的数据类型指基于数据的分布及其连接关系的类型(孔倩倩,2011)。

1. 狭义的数据类型

在可视化方法中,按照数据自身的特点来分,数据一般分为三种:标量数据、矢量数据和张量数据。标量数据亦称“无向量”,是指可以用一个不依赖坐标系的数字表征其性质的量,它只具有数值大小,而没有方向。典型的标量数据如温度、质量、体积、密度等。向量资料是指既有大小又有方向的量。常见的向量数据有速度、动量、冲量、加速度等,典型的向量资料如海洋风场和海洋洋流场资料。张量数据是指由若干坐标系改变时满足一定坐标转化关系的有序数组成的集合。

2. 广义的数据类型

海洋观测数据或者模式分析得出的科学数据通常都是离散数据点。通常来说,离散资料是科学计算可视化的主要研究对象之一。这种数据之间的连接关系的类型主要包括以下几种:

(1) 结构化数据

在海洋研究中,结构化数据是指能够在内存中组织成三维数组的空间离散数据。这些数据一般通过元素的行号、列号和层号来进行索引。

(2) 非结构化数据

非结构化数据是相对于结构化数据提出来的,它一般由一系列的单元,比如六面体、四面体、三棱锥等组成。非结构化数据的特点是在数据结构中须给出点的连接和位置关系,一般用链表、树等形式组织。

(3) 混合型资料

将前面提到的结构化数据和非结构化数据结合起来使用,便形成了一种新的方便的数据类型——混合型数据。

常用的数据存储格式是 NetCDF 格式,NetCDF 全称 Network Common Data Form,是一种面向数组的非常适合科学数据共享的一种自说明的,跨平台的数据存储交换格式(查石祥,2004)。它提供了专门的资料操作程序设计界面库和多种数据处理工具,可以方便的进行科学数据的存储分发。海洋流场矢量数据数据量巨大但结构比较简单,非常适合面向数组的 NetCDF 格式存储。

2.1.2 NetCDF 概述和预处理

1. NetCDF 概述

NetCDF(网络通用数据格式)是一种设计用于支持科学数据的创建、访问和共享的文件格式。这种文件格式在大气和海洋研究领域广泛用于存储变量,例如温度、压力、风速和浪高。NetCDF 数据的特性包括:

- ◆ 自描述性,即 NetCDF 文件包括关于其中所含数据的信息,如捕获数据元素的时间以及使用的测量单位。
- ◆ 可移植性,或跨平台性,即在一种操作系统上创建的 NetCDF 文件通常可通过其他操作系统上的软件读取。
- ◆ 可扩展性,即可有效地读取一个大 NetCDF 文件的一个小子集,而无需读取整个文件。

2. NetCDF 资料的存储

NetCDF 文件中的数据以数组形式存储。例如:某个位置处随时间变化的温度以一维数组的形式存储。某个区域内在指定时间的温度以二维数组的形式存储。三维(3D)数据(如某个区域内随时间变化的温度)或四维(4D)数据(如某个区域内随时间和高度变化的温度)以一系列二维数组的形式存储。

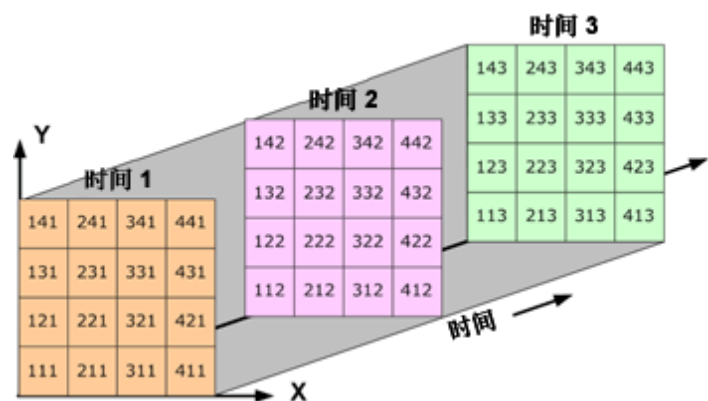


图 2-1 三维数据：某个区域内随时间变化的数据（<http://resources.arcgis.com>）

3. NetCDF 文件的基本组成部分

NetCDF 文件包含维度（dimensions）、变量（variables）和属性（attributes）三种描述类型，每种类型都会被分配一个名字和 ID，这些类型共同描述了一个数据集，NetCDF 库可以同时访问多个数据集，用 ID 来识别不同数据集。变量存储实际数据，维度给出了变量维度信息，属性则给出了变量或数据集本身的辅助信息属性，又可以分为适用于整个档的全局属性和适用于特定变量的局部属性，全局属性则描述了数据集的基本属性以及数据集的来源。将这几部分结合起来使用可以说明数据以及面向数组的数据集中各数据字段之间关系的含义。下图显示的是采用 CDL（网络公用数据格式语言）表示法来描述的 NetCDF 文件结构。CDL 是用于描述 NetCDF 文件内容的 ASCII 格式（ArcGIS Resources, 2017）。

```
netcdf filename {
dimensions:
    lat = 3 ;
    lon = 4 ;
    time = UNLIMITED ; // (2 currently)

variables:
    float lat(lat) ;
        lat:long_name = "Latitude" ;
        lat:units = "degrees_north" ;
    float lon(lon) ;
        lon:long_name = "Longitude" ;
        lon:units = "degrees_east" ;
    int time(time) ;
        time:long_name = "Time" ;
        time:units = "days since 1895-01-01" ;
        time:calendar = "gregorian" ;
    float rainfall(time, lat, lon) ;
        rainfall:long_name = "Precipitation" ;
        rainfall:units = "mm yr-1" ;
        rainfall:missing_value = -9999.f ;

// global attributes:
    :title = "Historical Climate Scenarios" ;
    :Conventions = "CF-1.0" ;

data:
    lat = 48.75, 48.25, 47.75;
    lon = -124.25, -123.75, -123.25, -122.75;
    time = 364, 730;
    rainfall =
        761, 1265, 2184, 1812, 1405, 688, 366, 269, 328, 455, 524, 877,
        1019, 714, 865, 697, 927, 926, 1452, 626, 275, 221, 196, 223;
}
```

图 2-2 CDL 文件示例（<http://resources.arcgis.com>）

维度、变量和属性的名称由任意的字母数字字符串（包括底线和连字符）组

成, 并且以字母或底线开头。但是, 以底线开头的名称是供系统使用的保留名称。NetCDF 名称区分大小写。

➤ 维度

NetCDF 维度包含名称和大小。维度大小是一个任意的正整数。每个 NetCDF 档中只有一个维度的大小是“无限制”的。这类维度是无限维度或记录维度。无限维度的变量可以沿着该维度增加到任意长度。维度可以用来表示实际物理维度, 例如时间、经度、纬度或高度。维度还可以用来为其他量建立索引, 例如, 网站或模型的运行数。

➤ 变量

变量代表相同类型的值数组。变数用来存储 NetCDF 文件中的大部分数据。变量具有名称、数据类型以及在创建该变量时指定的维度列表所描述的 shape。维度数称为秩 (或维数)。标量变数的秩为 0, 向量的秩为 1, 矩阵的秩为 2。变量还可以具有能够在变量创建后进行添加、删除或更改的相关属性。

➤ 属性

NetCDF 的属性用于存储辅助数据或元数据。大部分属性提供有关特定变量的信息。这些属性由变量名称与属性名称共同标识。提供有关整个 NetCDF 文件的信息的属性称为全局属性。这些属性由属性名称与空变量名 (CDL 中) 或特殊空变数 ID (C 或 Fortran 中) 共同标识。

4. NetCDF 数据预处理

本研究所需的实验数据为由美国国家气象局维护的全球预报系统 (GFS), 天气与海洋数据由数值模型生成, 预测每天生成四次, 可从 NOMADS (NOAA 操作模型归档和分发系统, NOAA Operational Model Archive and Distribution System) 网站下载。这些文档是 GRIB2 格式, 包含超过 300 条记录, 如图 2-3。

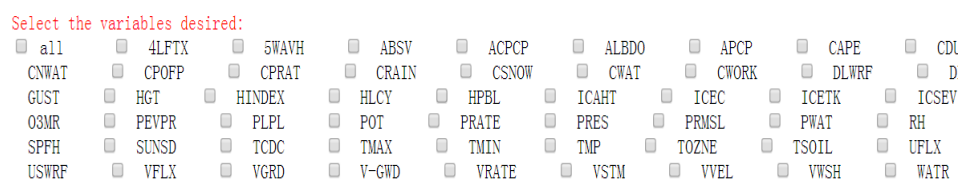


图 2-3 包含多种类的 NetCDF 文件

我们只需要这些记录中的几个来可视化特定等压在线的风力数据。使用由 Java 和 NetCDF-Java 类库开发解析 netCDF 的译码器 (netCDF-Java GRIB decoder), 这个基于 Java 的 NetCDF 译码器是美国大学大气研究协会 (University Corporation for Atmospheric Research, UCAR) 主持的 Unidata thredds 项目的一部分。本研究基于此译码器, 对包含海洋矢量场数据的 NetCDF 文件进行解析, 并提取出所需的海洋风场和洋流场数据, 最后, 将其转换为 Web 应用下较为通用且方便使用

的 JSON 格式，其中包括地理坐标，矢量的方向和矢量的大小，如图 2-4。

表 2-1 NetCDF-Java 类库操作 NetCDF 文件接口表

名称	功能	名称	功能
<code>ncdf_open()</code>	打开一个 NetCDF 文件	<code>ncdf_attname()</code>	返回一个属性名称
<code>ncdf_close()</code>	关闭一个 NetCDF 文件	<code>ncdf_create()</code>	创建一个 NetCDF 文件
<code>ncdf_varid()</code>	返回一个变量标识符	<code>ncdf_dimdef()</code>	创建一个维度
<code>ncdf_varget()</code>	读取一个变量	<code>ncdf_vardef()</code>	创建一个变量
<code>ncdf_attget()</code>	读取一个属性	<code>ncdf_attput()</code>	写入属性数据
<code>ncdf_inquire()</code>	返回文件信息	<code>ncdf_control()</code>	开始或介绍定义模式
<code>ncdf_varing()</code>	返回变量信息	<code>ncdf_varput()</code>	写入变量数据

```
[{"header":{"discipline":0,"disciplineName":"Meteorological products","gribEdition":2,"gribLength":131858,"center":7,"centerName":"US National Weather Service - NCEP (WMC)","subcenter":0,"refTime":"2014-01-31T00:00:00.000Z","significanceOfRT":1,"significanceOfRTName":"Start of forecast","productStatus":0,"productStatusName":"Operational products","productType":1,"productTypeName":"Forecast products","productDefinitionTemplate":0,"productDefinitionTemplateName":"Analysis/forecast at horizontal level/layer at a point in time","parameterCategory":2,"parameterCategoryName":"Momentum","parameterNumber":2,"parameterNumberName":"U-component_of_wind","parameterUnit":"m.s-1","genProcessType":2,"genProcessTypeName":"Forecast","forecastTime":3,"surface1Type":103,"surface1TypeName":"Specified height level above ground","surface1Value":10,"surface2Type":255,"surface2TypeName":"Missing","surface2Value":0,"gridDefinitionTemplate":0,"gridDefinitionTemplateName":"Latitude_Longitude","numberPoints":65160,"shape":6,"shapeName":"Earth spherical with radius of 6,371,229.0 m","gridUnits":"degrees","resolution":48,"winds":"true","scanMode":0,"nx":360,"ny":181,"basicAngle":0,"subDivisions":0,"lo1":0,"la1":90,"lo2":359,"la2":-90,"dx":1,"dy":1},"data":[-4.76,-4.75,-4.73,-4.72,-4.7,-4.68,-4.66,-4.64,-4.61,-4.59,-4.56,-4.53,-4.5,-4.47,-4.44,-4.41,-4.37,-4.34,-4.3,-4.26,-4.22,-4.18,-4.14,-4.09,-4.05,-4,-3.95,-3.9,-3.85,-3.8,-3.75,-3.7,-3.64,-3.59,-3.53,-3.47,-3.41,-3.35,-3.29,-3.23,-3.17,-3.1,-3.04,
```

图 2-4 解码后，提取的 JSON 格式的海洋矢量场数据
NetCDF 文件预理解析的流程图如下图所示：

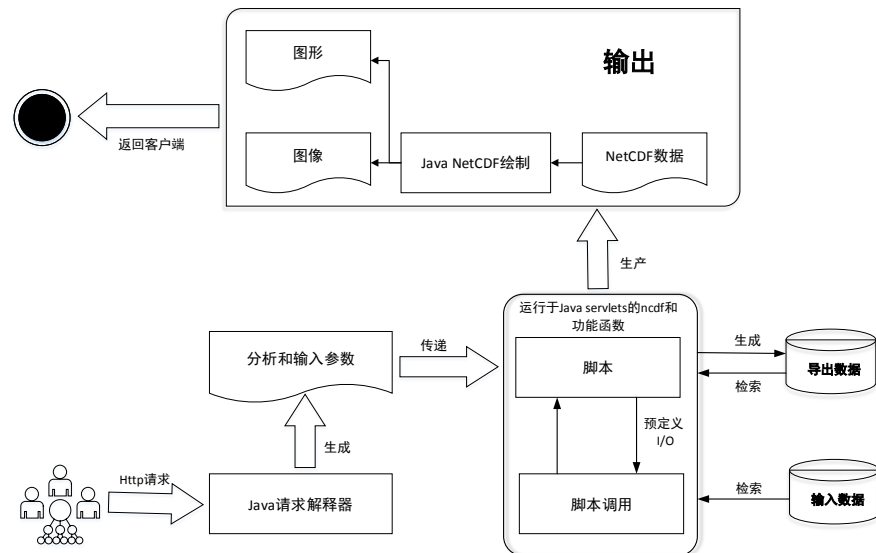


图 2-5 NetCDF 文件预处理流程示意图

2.2 海洋矢量场数据可视化的主要方法

运用科学计算可视化将各类海洋环境场数据转换为直观的图形图像呈现在研究者面前，易于观察、分析和理解。针对不同空间维度海洋环境数据的可视化表达可以采用不同的方法（刘健 等，2014）：一维数据主要是通过图形、符号，在空间场景上标识出该位置的数值信息；二维标量场数据的可视化方法有等值线图、区域填充图、基于网格的数据填图方法，二维矢量场数据的可视化方法有基于几何形状、基于颜色、光学特性和基于纹理的表达方法；三维矢量场（三维流场、风场资料）常见的是基于几何形状的剖面重构法、流线法，粒子系统法。

具体来说，海洋矢量场数据可视化的目的在于运用计算机图形学，数字图像处理，数据挖掘等多种方法和技术将通过生产时间和实际测量得到的矢量场数据，真实准确的可视化出来，并支持修改可视化算法参数进行结果交互处理。按照向量场数据的可视化方式，向量场可视化一般可分为以下几类：点图标法、向量线、纹理法和特征法，粒子系统法（周迪斌，2008）。

2.2.1 点图标法

点图标是最简单直观的矢量场表示方法，经常使用箭头、锥体、有向线段等像素来表示流场的方向和大小。在二维空间中，箭头可以较好的反映出向量大小和方向信息，但是放大后箭头之间可能相互覆盖，而且在三维的显示中，普通箭头方法容易产生二义性。如点图示过多、过于密集易导致图示迭置、杂乱无章，而太少又不能准确地表达向量场的变化情况，无法揭示出数据的内在连续性，因此比较适合局部可视化。

2.2.2 向量线法

向量线法源于实验型流场的可视化技术。与实验方法类似,向量线包括流线、时线、脉线、迹线。流线一般用于稳定向量场,而其它方法一般用于非稳定流场。向量线计算可以采用数值积分方法,数值计算可以在物理空间进行,也可在计算空间进行。与点图示法相比,向量线能表示场的连续性,绘制效果较好,但是绘制质量严重依赖于初始点的位置选取,选择不好可能会遗漏向量场重要的特征和细节,并且其表示效果是静态的,无法动态的反应向量场数据的动态变化情况。

2.2.3 特征法

特征法一直是流场可视化研究的一个热点区域(Helman et al.,1990)。特征是向量场中用户感兴趣的区域,特征提取是数据挖掘中的含义,在大量繁杂冗余的数据中能够提取出隐藏的,用户感兴趣的数据,或者是去掉用户不感兴趣的区域。流场特征包含两个部分:(1)流场中有意义的形状、结构、变化和现象,如涡流、激波等;(2)从数据场中分离出来的用户感兴趣的区域。特征法就是通过对场中这些特征提取,减少可视化结果冗余,而保持海洋特有的流场现象。在特征可视化中,通过提取特征的过程,得到一种可以改进原有抽象可视的表示,帮助用户自动忽略许多冗余的不感兴趣的数据或自动提取用户感兴趣的区域。

2.2.4 粒子系统法

粒子方法是利用光强、颜色信息来进行矢量场可视化的,可用于模糊对象的绘制,表示出不规则的复杂几何形体,一般用于流场、波浪等向量场的可视化中。粒子束方法的基本单元是粒子。每个粒子同时具有物理属性和图像属性,它的形状可以是点、线、四边形、六边形。每个粒子具有一定的生命周期,即每个粒子可以存活的时间。所有的粒子具有大小和颜色属性。向量场可视化中可将粒子的某一具体性质与向量场中的向量联系起来。在模拟物理海洋中的流场时,可将速度向量映像为粒子运行的动态性质,在粒子生命周期内赋予粒子属性。根据粒子的性质可分为点粒子跟踪法和面粒子法。点粒子可看作是一发光的点状质点,其运动轨迹是粒子在不同时刻在流场中的位置来组成。面粒子是造型小、能反射有向光源的面片(王想红,2013)。

综合以上分析来看,在实现矢量场动态可视化效果和实现难易程度的综合考虑下,粒子方法是一个较为理想的方法,在矢量场可视化中可将粒子的某一具体性质与矢量场中的向量联系起来,能够得到较为理想的可视化效果。粒子系统方法可以说是一种基于物理模型来解决问题的方法,它的核心不在于如何显示图形,而是在于对某个物理模型的理解和分析,即对微小物质模型的规则提取,粒子的

运动规律可以很简单也可以很复杂,这取决于模拟的物理模型的复杂程度。采用动画形式的粒子运动能真实、有效地可视化流场变化。固定显示刷新时间和时间步长,粒子不同时间所处的空间位置直接表示了海洋风场中的速度变化。

2.3 基于 WebGL 的 SPH 实时流体模拟方法

计算流体动力学 (Computational Fluid Dynamics, CFD) 是计算机图形学研究中的热门话题,它通过数值计算的方式对流体力学的控制方程进行求解,从而预测流体的流动,很多国内外相关的研究人员都专注于开发新的和更好的方法来模拟和可视化流体 (Micky, 2006), 很多领域,如计算机游戏,虚拟手术和自然现象的模拟都对更实时更快速的仿真有很大的需求。然而,流体模拟的实时计算负担是非常高的,特别是当模拟自由表面流动时, (Harada, 2007) 计算量是流体实时模拟中一个主要的阻碍因素,所以将流体模拟应用到实时的应用程序中是相当困难,因此自由表面流动的实时仿真模拟是目前一个开放的研究领域。

2.3.1 光滑粒子流体动力学 (Smoothed Particle Hydrodynamics)

Navier-Stokes 方程是描述流体运动的最基本力学方程组,在一定假设条件下对 N-S 方程的简化处理,可衍生出许多满足实际工程需要的应用方程,因此对 N-S 程的求解是十分必要的,以下为 N-S 方程的一种表示方式(陈英建, 2012):

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} \right) = -\nabla p + \mu \nabla^2 \mathbf{v} + \rho \mathbf{g} \quad (2-1)$$

式中, $\mathbf{v}$ 为流体速度, ρ 为流体密度, μ 为流体的粘度系数, ∇ 为梯度操作数, ∇^2 为拉普拉斯操作数, $\mathbf{g}$ 为作用于流体的所有外力密度场。

光滑粒子流体动力学 (Smoothed Particle Hydrodynamics, SPH) 方法就是求解的一种方法 (丁亮, 2013), 它是模拟流体流动的一种无网格自我调整拉格朗日粒子法, 在拉格朗日法中, 将流体系统想象成一个封闭空间, 即使流体在流动过程中发生膨胀、压缩、变形等变化, 流体的总质量也会保持不变。SPH 方法的基本思想, 就是将连续的流体想象成一个个相互作用的微粒, 这些粒子相互影响, 共同形成了复杂的流体运动, 如图 2-7 所示。对于每个单独的流体微粒, 依旧遵循最基本的牛顿第二定律: $\mathbf{F} = m\mathbf{a}$, 同时, 在 SPH 算法里, 流体的质量是由流体单元的密度决定的, 所以一般用密度代替品质, 即为 $\mathbf{F} = \rho\mathbf{a}$ 。在 SPH 方法中, 每一个流体粒子都受三个作用力, 分别为由流体内部的压力差产生的作用力、粒子之间速度差引起的粘度力和外部力, 外部力一般就是重力。

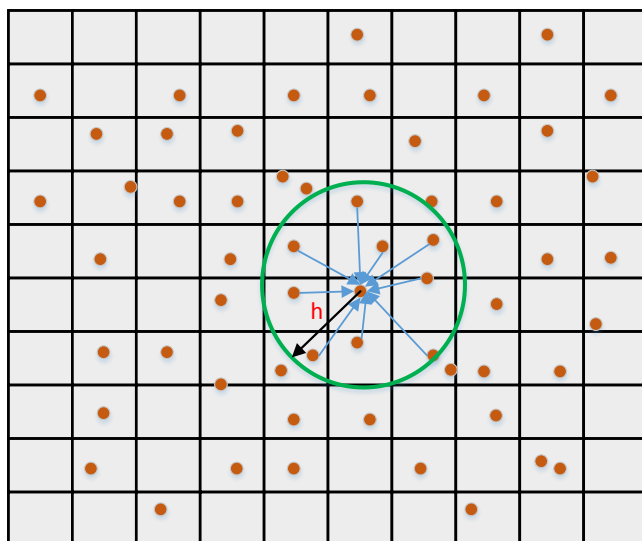


图 2-7 光滑粒子搜索半径内所有粒子对当前粒子作用力示意图

用公式 n 来代表粒子受力，公式 n 为其展开形式。

$$\mathbf{F} = \mathbf{F}^{External} + \mathbf{F}^{Pressure} + \mathbf{F}^{Viscosity} \quad (2-2)$$

$$\rho \mathbf{a} = \rho \mathbf{g} - \nabla p + \mu \nabla^2 \mathbf{v} \quad (2-3)$$

其中， $\mathbf{F}_i^{Pressure}$ 是由流体内部的压力差产生的作用力，正是由于液体内部压力差的存在，液体才会源源不断的流动，在数值计算上，它等于压力场的梯度，方向由压力高的区域指向压力低的区域

$\mathbf{F}^{Viscosity}$ 是由粒子之间的速度差引起的，运动的流体内部由于压力差的存在而分为快速流动的部分和速度较慢的部分，那么速度较快的部分就会挟带和刮蹭着速度较慢的部分，这个力的大小跟流体的粘度系数 μ 以及速度差有关。

$\mathbf{F}^{External}$ 被称为外部力，一般指的是重力。

$$\mathbf{F}^{External} = \rho \mathbf{g} \quad (2-4)$$

其中 $\mathbf{g}$ 为重力加速度。

参考 Matthias Muller 和 Kenneth 公式推导结果对公式 n 进行展开，得到：

$$\mathbf{F}_i^{Pressure} = -\nabla p(r_i) = -\sum_j m_j \frac{p_i + p_j}{2\rho_j} \nabla W(r_i - r_j, h) \quad (2-5)$$

$$\mathbf{F}_i^{Viscosity} = \mu \nabla^2 \mathbf{v}(r_i) = \mu \sum_j m_j \frac{v_j - v_i}{\rho_j} \nabla^2 W(r_i - r_j, h) \quad (2-6)$$

(2-14)即为方程(2-1)的简化形式，也就是 Navier-Stokes 方程的一个简单形式，以上的过程即为 N-S 方程的一个简单的推导过程，全面的 N-S 方程有更加严格的形式和推导过程，不过核心思想都已经通过上面的推导表达了出来。

通过以上分析可知，流体是由一个个分散的粒子组成，但同时流体是连续的充满整个空间的，流体中每个位置参与运算的数值，都是由周围一组粒子的相互作用累加起来的。所以对于整个粒子空间中粒子的受力，也是由各个位置所受的

作用力累加而得到。

2.3.2 基于 WebGL 的实时流体模拟方法原理

WebGL 可以直接在 HTML5 的 Canvas 元素中绘制三维动画并提供硬件三维加速渲染,是基于 OpenGL ES2.0 标准,并使用 OpenGL 着色语言 GLSL,是一个用来在 Web 上生成三维图形效果的应用程序设计界面。利用 WebGL 进行流体模拟存在三个主要问题,分别为邻域粒子的查找及作用力的计算,中间数据的交换与存储和计算结果实时渲染。在 OpenGL 中,可以利用 C++语言的优势,使得邻域粒子的查找和作用力的计算在 CPU 中执行,且能开启多个线程,但 Web 端应用程序的运行环境较为特殊,是单线程执行的。这里包含两层含义,其一 JavaScript 代码的执行是单线程,其二渲染进程和代码执行进程是互斥执行的,那么整个浏览器的资源都去运行大量的计算任务时,页面就会处于假死状态,页面的渲染帧率会降到最低,这样整个前端应用程序就失去了响应。此时的解决方案也必然是将大部分计算放在 GPU 中执行,而利用 WebGL 中其与 GPU 交互的方式,即整个图形流水线,可以实现计算产生的中间数据的存取都以纹理缓冲区中的 RGBA 方式存储,而不用频繁的进行 CPU 和 GPU 之间的数据传输。同时,本文采用离屏渲染的方式,将多次需要渲染的数据写入帧缓冲区中,从而实现一次性写入主帧缓冲区进行渲染。整个方法流程示意图如图 2-8 所示。

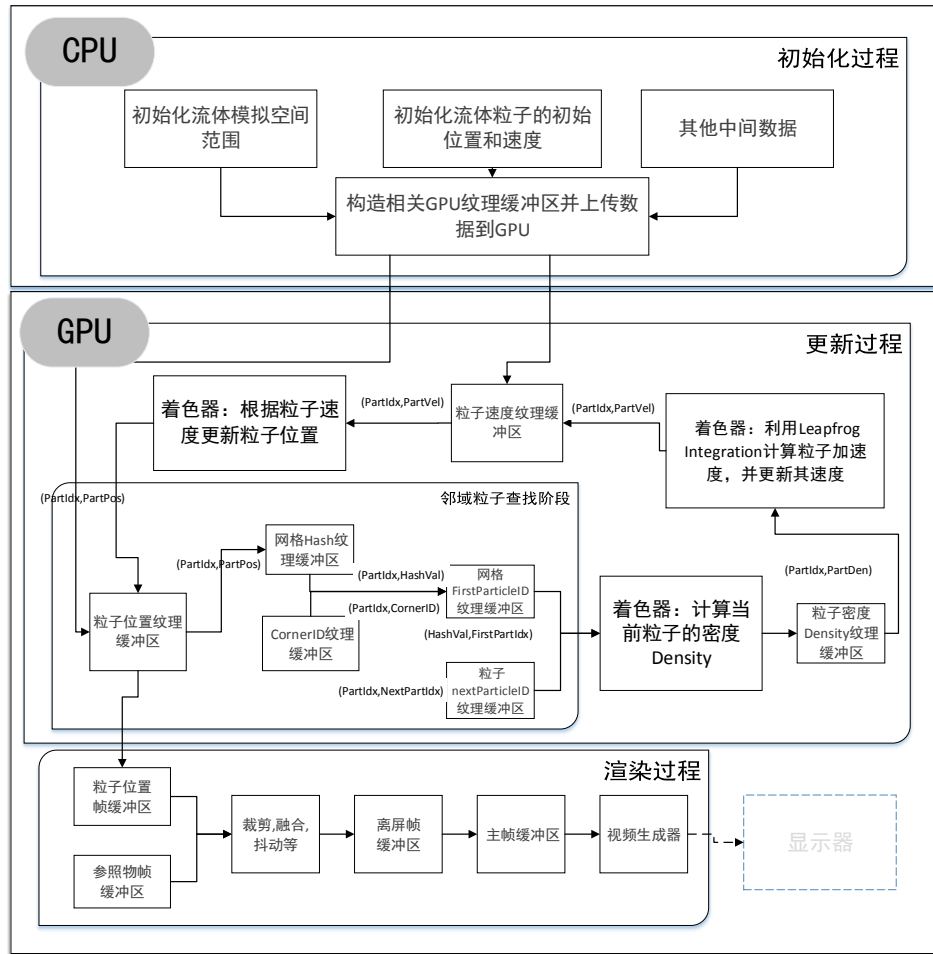


图 2-8 基于 WebGL 的 SPH 流体模拟方法示意图

2.3.3 方法实现过程

1、初始化过程

初始化时，首先利用两组顶点和片元着色器建立两组组包含纹理缓冲区的空白帧缓冲区，且纹理的尺寸均为 1024 规格大小，分别用来存储所有粒子的位置和速度。如下图 2-9 所示，其中 posX, posY 和 posZ 为粒子的坐标值，A 为纹理的透明位，设置为 1.0，将其赋值给 gl_FragColor，而纹理中的点的位置 gl_Position 设置为 (vertex2DIndexX, vertex2DIndexY, 0.0, 1.0)，其中 vertex2DIndexX 和 vertex2DIndexY 用来确定在二维纹理中点的位置，且都与当前粒子在所有粒子中的 ParticleID 相关。粒子速度的帧缓冲区与位置帧缓冲区相同，均能通过粒子的 ParticleID 找到其在纹理存储的值。

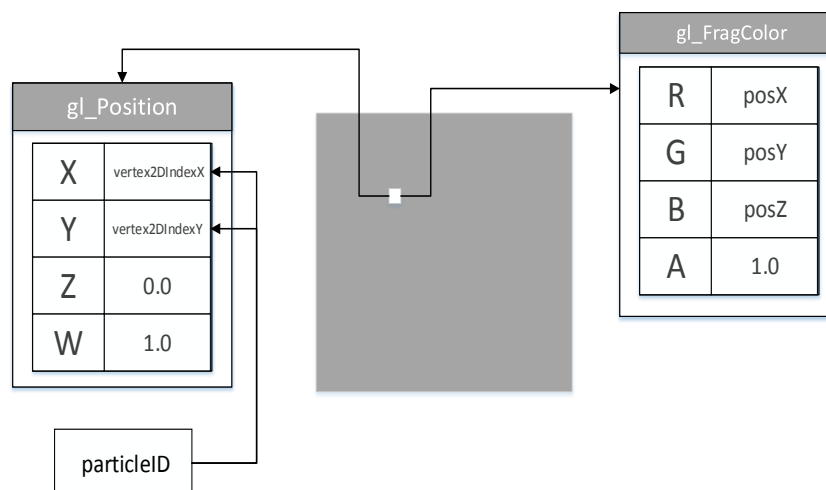


图 2-9 粒子位置纹理中坐标与颜色值关系示意图

上文提到将流体模拟的空间进行网格体素化,这里同样也需要初始化一个与位置帧缓冲区类似的映像粒子所属网格的帧缓冲区,而此时纹理的 `gl_FragColor` 赋值为 $(0.0, 0.0, 0.0, \text{HashVal})$, `HashVal` 即为根据粒子的位置和划分的网格的关系,经过 Hash 函数计算所得的 Hash 值,为了符合纹理的透明位的值的要求,本文对其进行了 0.0-1.0 的处理。同时,在确定邻域网格之后,遍历某个网格内多个粒子时,这时需要建立一个存储多个粒子中第一个粒子的 `ParticleID` 的帧缓冲区,此过程需要在 GLSL 语言中对 `VoxelID` 进行排序,目前存在很多快速的排序算法在 GPU 上得到实现,如归并排序 (Matt, 2005),奇偶归并排序 (Randima, 2005) 和快速排序 (Cedeman, 2009) 等等,本文采用快速排序方法。纹理中 `gl_FragColor` 赋值为 $(\text{firstParticleID}, 0.0, 0.0, \text{nextParticleID})$,如果是多个粒子中的最后一个粒子, `nextParticleID` 设置为 -1, `gl_Position` 赋值为 $(\text{VoxelIDX}, \text{VoxelIDY}, 0.0, 1.0)$,这样就可以快速的查找到当前粒子的所有在搜索半径内的粒子。

2、更新过程

更新过程主要分两部分,分别为确定邻域粒子和计算所有光滑搜索半径内的粒子对当前粒子的影响作用,并更新其位置和速度。本文采用改进的链表搜索法来确定邻域粒子,链表搜索法对于给定的作用域范围,将其用网格进行划分,这个过程称为体素化,每一个网格被指定一个序列编号,同时每一个粒子都可以根据其自身的坐标位置计算出粒子所在的网格 (Dong et al., 2015)。网格的大小可以设置粒子的光滑搜索半径 h ,也可以设置为 $2h$,不同的网格大小决定搜索的网格的个数,但整体的思路是一致的,本文设置网格的大小为 $2h$ 。同时,根据上述粒子和网格配对情况,利用 `VoxelID` 和 `ParticleID` 之间的关系建立 Hash 函数,并生成排序的 Hash 表。在搜索某一粒子的所有邻域粒子时,根据其坐标位置确定其所在网格,然后确定其邻域 8 个网格,根据 Hash 表找到 8 个网格内部所有

的粒子,并判断其是否在光滑搜索半径内,如果在再加入邻域粒子表中,留待之后计算下一帧中粒子所在的位置和速度。在每一帧的更新中,执行以上搜索邻域粒子的过程,使得这些粒子可以逼真连续地模拟流体的流动状态,其搜索流程如下图所示:

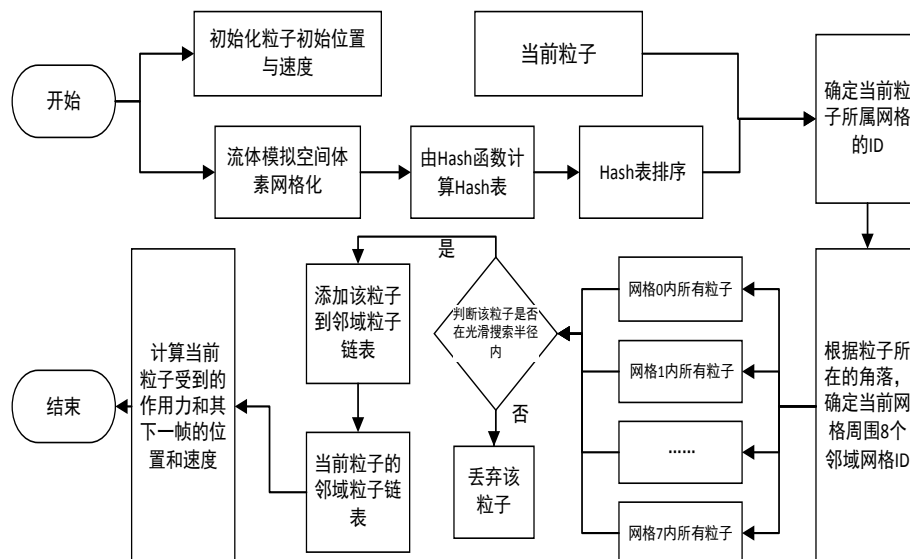


图 2-10 链表搜索算法流程图示意图

上述搜索流程中,通常会出现一个网格中含有多个粒子的情况,这时常见做法是将另外建立一个表用来存储多个粒子,而主 Hash 表只存储多个粒子的第一个粒子或者不做区分,将多个粒子全部放到主 Hash 表中,这也和算法实现的语言的不灵活有很大关系,如 OpenGL 中的 C++。在 WebGL 中,其主要语言之一为 JavaScript,本文利用 JavaScript 语言的动态添加属性的灵活特性对粒子对象添加 next 属性,用来存储下一个粒子的 ID。针对主 Hash 表来说,其只需要存储同一网格的第一个粒子即可,并且不用建立新表,程序会根据粒子对象的 next 属性找到同一网格内的下一个粒子对象。另外,在确定邻域 8 个网格时,往往根据粒子位置实时计算其位于网格立方体空间八个角落的哪一个角落,并且每一个粒子均需要这类计算。每个立方体模拟空间都可以沿着 XYZ 三个方向的一半进行分割,这时可以得到类似八叉树的结构,即为 $2*2*2$ 的一个网格。因此,本文针对这种情况,在初始将粒子分配到各个网格并计算 Hash 表时,添加一个 (ParticleID, CornerID) 表,其中 CornerID 代表粒子位于网格立方体空间的 8 个角落的编号,其值为 0-7,分别代表 8 个角落。这样不仅可以减少确定邻域 8 个网格的重复计算,并且利用这个表作为索引可以快速确定某粒子所在网格的 8 邻域网格。

根据初始化过程中初始化的三个帧缓冲区所包含的信息及其关系,可以确定当前粒子光滑搜索半径内所有粒子。此时,为了使用 GPU 的并行特性,在计算

粒子下一帧所处的位置和速度时,需要三对顶点着色器和片元着色器来分别计算当前粒子的密度 (Density), 速度 (Velocity) 和位置 (Position), 其中的数据传输流程示意图如图 2-11。

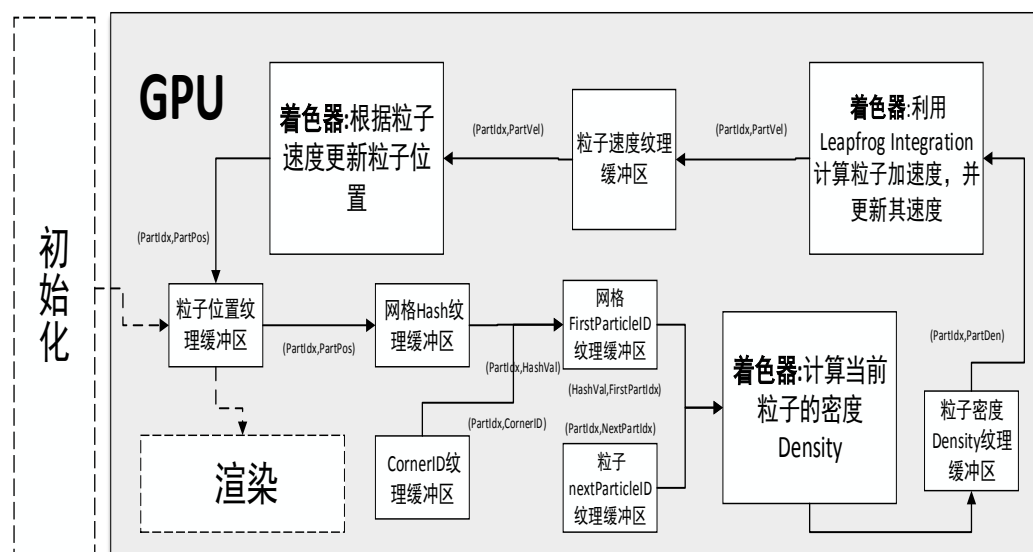


图 2-11 粒子位置更新过程中数据传输示意图

上述数据传输流程需要新建一个存储粒子密度信息的帧缓冲区,与粒子位置帧缓冲区类似,通过 ParticleID 最后索引到其密度信息,然后传输到计算粒子速度的着色器中,将计算结果更新到粒子速度帧缓冲区中。最后在计算粒子下一帧位置的着色器中,利用 LeapFrog 方程整合所计算的数据,得到最后所有粒子的位置,更新到粒子位置帧缓冲区。另外,在对流体表面信息进行提取时,本文采取 Marching Cube (Dong et al., 2015) 来构造水表面,其主要思想是:已知一个空间中的数据场,并给出一个期望的资料值,以确定要重构的等值面。将空间划分为小立方体网格 (cubes), 计算 Cube 的顶点数据值(Øystein, 2010)。该算法简单易实现,是目前常用的流体表面重构方法。

3、渲染过程

渲染过程包括抽取粒子位置帧缓冲区中位置坐标信息并交给主帧缓冲区,获取场景中的其他待绘制物体的缓冲区信息和整个场景的光照信息等等。在实际流体模拟中,流体会与周围物体进行碰撞交互,这时如果想要逼真的模拟效果,需要将流体周围其他物体的信息考虑在内。在更新下一帧粒子位置时将边界信息对粒子速度和位置的影响作用写入着色器中,然后这部分的信息交给主帧缓冲区。最后,待绘制数据更新完毕,直接将主帧缓冲区内容交给 GPU 绘制到屏幕上即可。

在最后渲染过程中,本文使用了离屏渲染 (OffScreen Rendering) 技术,指的是 GPU 在当前屏幕缓冲区以为新开辟一个缓冲区进行渲染操作。根据之前多

次计算的结果,将所有需要绘制的数据结果存储到一个帧缓冲区,然后一次性绘制,减少了多次渲染所耗费的计算资源。本文使用的这个方法,可以有效解决本节开始提到的几个问题,将大部分计算全部放在 GPU 中执行,这在 Web 三维应用程序苛刻的运行环境下,如单线程和数据传输耗时等,大大减轻了 Web 前端程序的计算任务负担。同时中间的计算结果全部在 GPU 中存取,大大减少了 CPU 和 GPU 的数据交换带来的损耗,减轻了 CPU 的负担,能够有效提高渲染的帧率。

2.3.4 实验结果与对比分析

SPH 流体模拟中的邻域粒子查找算法,可采用全搜索算法,链表算法和树形算法,经过综合对比,本文采用链表搜索算法进行在 WebGL 环境下进行 SPH 流体的模拟,并针对算法过程中确定当前粒子邻域网格时的冗余计算,提出建立粒子在网格中位置编号查找表,最后提出一个在 WebGL 中进行 SPH 流体模拟的方法。本文的实验平台所使用的操作系统为 Window 7 64 位,处理器为 Intel Core i5 2.7GHz,安装内存 8GB,显卡为 Intel Iris Graphics 6100 1536MB。文中采用基于全搜索算法的 CPU-GPU 流体模拟,基于链表搜索方法的 CPU-GPU 流体模拟和 Bayraktar 算法的 GPU 流体模拟,并与文中提出的基于 WebGL 的全 GPU 方法渲染帧率比较,如图 2-12 所示。

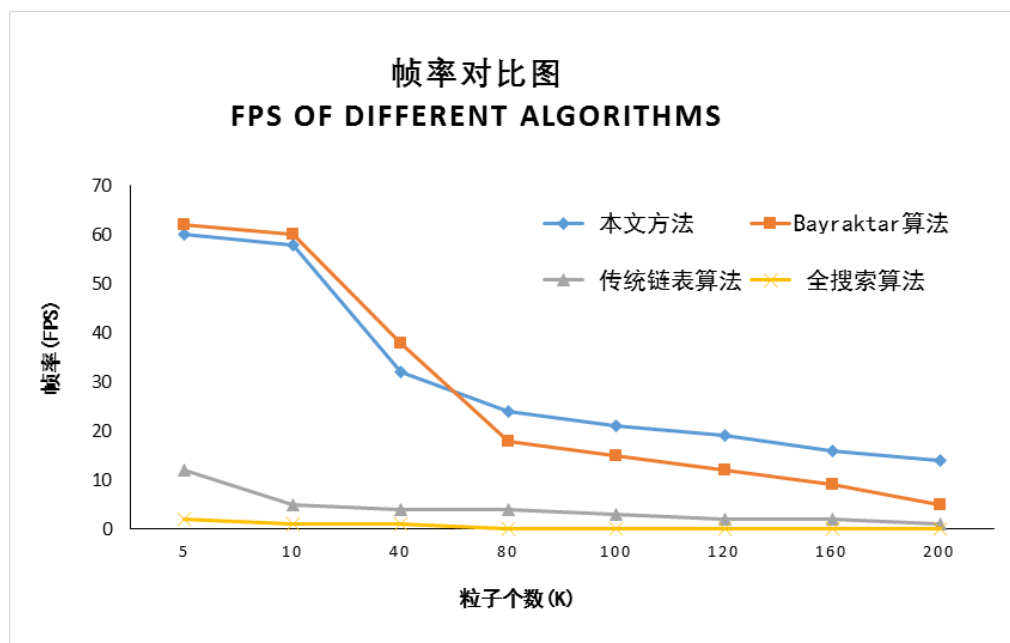


图 2-12 不同数目的粒子在不同算法下渲染时的帧率对比图

从图 2-12 可知: (1) 基于 GPU 的邻域粒子查找算法的性能明显优于基于 CPU 的邻域粒子查找算法; (2) 随着粒子规模的不断增大,基于 GPU 的邻域粒子查找算法的渲染帧率迅速下降,这在 Web 应用程序中尤为明显,这是由于 Web

应用程序在浏览器中计算和渲染是单线程进行的;(3)本文提出的在 WebGL 中,基于多个帧缓冲区的流体模拟方法在粒子数目较少时,不如经典的 Bayraktar 算法。但在粒子越来越多的情况下,本文提出方法在粒子数目 200K 的情况下仍能达到 15 帧的帧率,这明显的优于经典的 Bayraktar 算法的 5 帧。

下图 2-13 是进行的溃坝模拟,由数万个粒子模拟的水的立方体从高处下落,在触碰到边界时改变运动方向,并在模拟过程中放置障碍物观看流体的流动效果。其中,(a)是粒子模拟的水分子从高处落下时的初始状态,(b)(c)(d)均为粒子模拟的水分子与边界(墙壁,障碍物)碰撞产生的运动效果,(e)(f)是对流体表面信息进行提取,模拟真实的水表面,流体的自由表面采用文献(Dong et al., 2015)中 Marching Cube 算法构建。

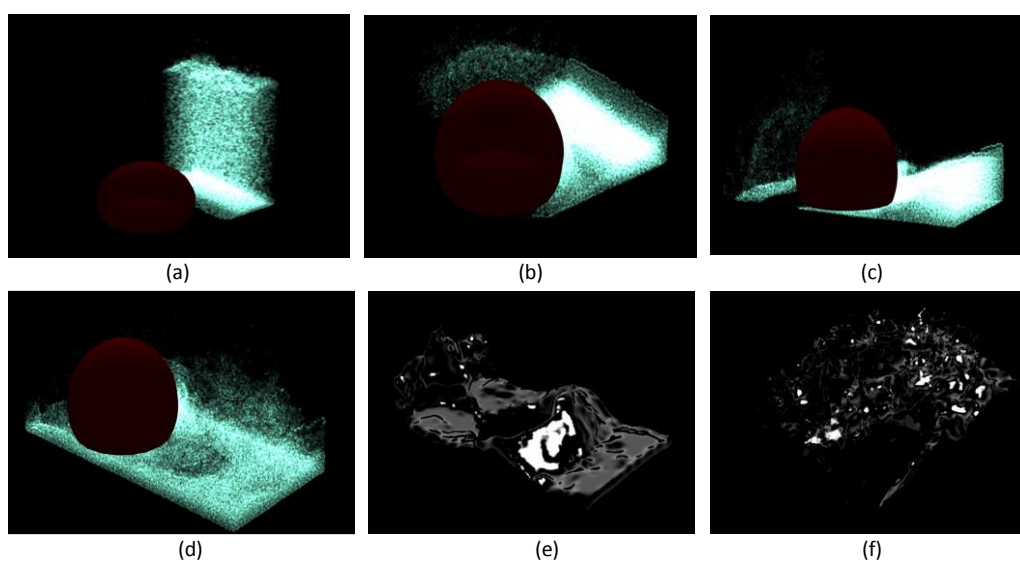


图 2-13 溃坝流体模拟粒子与水流效果示意图

2.4 SPH 原理简化的海洋矢量场数据动态可视化方法

与标量场相比,矢量场最大的不同点在于每个物理量不仅具有大小,而且具有方向,因此,矢量场的可视化方法与标量场有所不同且更为复杂。矢量数据映射的目的是将预处理后的矢量数据转换为可通过图形形式显示的几何数据,这是矢量场可视化的核心。本文在上一章节基于 WebGL 的 SPH 实时流体模拟方法的基础上,提出利用 SPH 原理简化的方法来对海洋矢量场进行可视化,其中有如下几个关键问题需要考虑:

(1) 粒子化: 对所得的流体以粒子来表示,若问题域不是粒子形式的,则使用若干没有相互作用的粒子,使其分布于问题域空间。这就是 SPH 方法的无网格性,这一步主要是要保证粒子化后,问题域求解的稳定性,特别是当求解边界条件的问题时,由于粒子的任意分布,会带来一定的求解难度。

(2) **场函数的表示**: 将 SPH 的基本方程以积分法来表示, 也就是 SPH 方法中的核近似法, 由于积分法的光滑作用, 使得 SPH 方法的场函数在数学计算上具有较好的稳定性。

(3) **自适应操作**: 从 SPH 的基本特性可知, 在对 SPH 粒子初始化后, 粒子的运动 SPH 方程来自动控制, 就是因为实际模拟过程的每个时间步长内, 都要进行第 3 步的粒子近似过程, 由于不受随着时间变化粒子分布时刻变化的影响, 因而解决了流体模拟中大变形的问题 (陈英建, 2012)。

(4) **赋予真实属性**: 通过近似法对粒子求解之后, 对粒子赋予质量等各种属性, 使得粒子成为具有真实材料属性的粒子。

(5) **确定时间步长**: 通过动力学中传统的逐步积分法来确定流体粒子处理最优的时间步长, 来保证整个积分过程的稳定性, 并得到随着时间变化粒子位置的变化值, 从而进行粒子更新。

和其他流体力学中的数学方法类似, SPH 算法同样涉及到“光滑核”的概念, 可以这样理解这个概念, 粒子的属性都会“扩散”到周围, 并且随着距离的增加影响逐渐变小, 这种随着距离而衰减的函数被称为“光滑核”函数, 最大影响半径为“光滑核半径”。针对海洋矢量场数据进行动态可视化, 光滑核函数 also 具有重要的意义, 当系统不断对粒子系统中每个位置进行速度和位置更新时, 需要更多的考虑每个粒子的周围一些粒子对其影响, 这具有实际的意义, 物理现象中每个现象都不是孤立, 都是有相互影响的。同时, 寻找某粒子的周围粒子是有一定范围的, 超过一定范围的粒子对该粒子的影响作用非常小, 可以不做考虑。如果将此核函数范围设置的过大也会显著的影响场景的渲染效率。

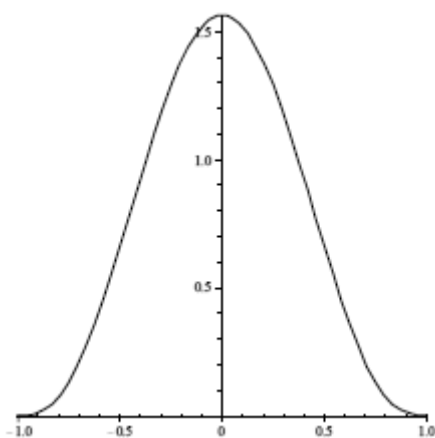


图 2-14 光滑核函数一般具有的形态

反之不难理解, 尽管我们将流体视为一个个分散的粒子, 但流体毕竟是连续充满整个空间的, 流体中每个位置参与运算的值都是由周围一组粒子累加起来的。在 SPH 方法中, 核近似函数由以下 2-7 所示方程式给出:

$$f(x) = \int_{\Omega} f(x') \delta(x - x') dx' \quad (2-7)$$

光滑核函数是一种无穷可导的函数形式，在 SPH 方法中光滑核函数（smoothing kernel function）可以保证函数在求解过程中的连续性，方程(3-2)就是一种简单的光滑核函数，。

海洋矢量场中的粒子近似

在 SPH 方法中，每个流体粒子都带有自己的独立属性，这些属性之间互不影响，如粒子的质量、密度、速度等属性，这就为粒子近似法的应用提供了可能。在粒子近似法中，首先将作用域内的粒子离散化，然后根据条件对一定区域内粒子的某种属性叠加求和。

下面就是 SPH 方法中粒子近似法的实现过程，首先是用粒子的体积 V 来代替在公式 2-7 的函数积分法中的体积积分 dx ，给定粒子 j 的质量 m_j 可用方程式 2-8 来表示。

$$m_j = \Delta V_j \rho_j \quad (2-8)$$

式中—— ρ_j 为粒子 j 的密度， $j=1,2,\dots,N$ ， N 是 x 域内粒子的总数量。

$$\begin{aligned} f(x) &= \int_{\Omega} f(x') W(x - x') dx' \approx \\ &= \sum_{j=1}^N f(x_j) W(x - x_j, h) \Delta V_j = \\ &= \sum_{j=1}^N f(x_j) W(x - x_j, h) \frac{1}{\rho_j} (\rho_j \Delta V_j) = \\ &= \sum_{j=1}^N f(x_j) W(x - x_j, h) \frac{1}{\rho_j} (m_j) = \end{aligned} \quad (2-9)$$

$$f(x) = \sum_{j=1}^N \frac{m_j}{\rho_j} f(x_j) W(x - x_j, h) \quad (2-10)$$

在函数积分法中，积分函数的近似表达式 $f(x)$ ，可以在粒子近似法中以方程式 2-9 的离散化的粒子近似式表示，方程式 2-9 如上所示。

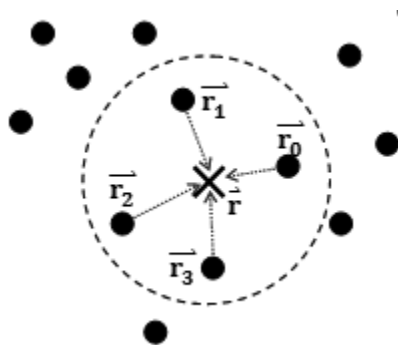


图 2-15 当前粒子周围粒子对其作用力示意图

同时, 设想流体中某点 $\mathbf{r}$ (此处不一定有粒子), 在光滑核半径 h 范围内有数个粒子, 位置分别是, $\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_j$, 则该处某项属性 A 的累加公式为:

$$A(\vec{r}) = \sum_{j=1}^N A_j \frac{m_j}{\rho_j} W(\vec{r} - \vec{r}_j, h) \quad (2-11)$$

其中 A_j 是要累加的某种属性, m_j 和 ρ_j 是周围粒子的质量和密度, r_j 是该粒子的位置, h 是光滑核半径。函数 W 就是光滑核函数, 光滑核函数两个重要属性, 首先一定是偶函数, 也就是 $W(-r)=W(r)$, 第二, 是“规整函数”, 也就是 $\int W(r) dr = 1$ 。

根据上一篇的公式 2-3, 对于粒子 i , 它的位置为 $\mathbf{r}_i$, 此处的加速度 $\mathbf{a}(\mathbf{r}_i)$ 的推导公式为:

$$\vec{a}(\mathbf{r}_i) = \vec{g} - \frac{\nabla p(\mathbf{r}_i)}{\rho(\mathbf{r}_i)} + \frac{\mathbf{u} \nabla^2 \mathbf{u}(\mathbf{r}_i)}{\rho(\mathbf{r}_i)} \quad (2-12)$$

也就是说, 需要逐个推出密度 ρ , 压力 p , 速度 $\mathbf{u}$ 相关的累加函数, 根据公式 2-11, 用密度 ρ 代替 A , 可以得到

$$\rho(\vec{r}) = \sum_{j=1}^N \rho_j \frac{m_j}{\rho_j} W(\vec{r} - \vec{r}_j, h) = \sum_{j=1}^N m_j W(\vec{r} - \vec{r}_j, h) \quad (2-13)$$

密度

计算使用的光滑核函数称为 Poly6 函数, 具体形式为:

$$W_{\text{poly6}}(\vec{r}, h) = \begin{cases} K_{\text{poly6}}(h^2 - r^2)^3, & 0 \leq r \leq h \\ 0, & \text{otherwise} \end{cases} \quad (2-14)$$

其中, K_{poly6} 是一个固定的系数, 根据光滑核的规整属性, 通过积分计算出这个系数的具体值, 在 2D 情况下, 在极坐标中计算积分:

$$K_{\text{poly6}} = 1 / \int_0^{2\pi} \int_0^h r(h^2 - r^2)^3 dr d\theta = \frac{4}{\pi h^8} \quad (2-15)$$

由于所有粒子的质量相同都是 m , 所以在 3D 情况下, $\mathbf{r}_i$ 处的密度计算公式最终为:

$$\rho(\mathbf{r}_i) = m \frac{315}{64\pi h^9} \sum_{j=1}^N (h^2 - |\vec{r}_i - \vec{r}_j|^2)^2 \quad (2-16)$$

压力

根据上一节的结论, 在位置 $\mathbf{r}_i$ 之处的由压力产生的作用力的计算公式为

$$\mathbf{F}_i^{\text{Pressure}} = -\nabla p(\vec{r}_i) = -\sum_{j=1}^N \rho_j \frac{m_j}{\rho_j} W(\vec{r} - \vec{r}_j, h) \quad (2-17)$$

不过, 这个公式是“不平衡”的, 也就是说, 位于不同压强区的两个粒子之间的作用力不等, 所以计算中一般使用双方粒子压强的算术平均值代替单个粒子的

压力

$$F_i^{Pressure} = -\sum_{j=1}^N \rho_j \frac{m_j(p_i+p_j)}{\rho_j} W(\vec{r} - \vec{r}_j, h) \quad (2-18)$$

对于单个粒子产生的压力 p , 可以用理想气体状态方程计算

$$P = K(\rho - \rho_0) \quad (2-19)$$

其中 ρ_0 是流体的静态密度, K 是和流体相关的常数, 只跟温度相关。

压力计算中使用的光滑核函数称为 Spiky 函数

$$W_{spiky}(\vec{r}, h) = \begin{cases} K_{spiky}(h-r)^3, & 0 \leq r \leq h \\ 0, & otherwise \end{cases} \quad (2-20)$$

$$\text{在 3D 情况下, } K_{spiky} = 15/(\pi h^6) \quad (2-21)$$

现在可以整理出公式 2-12 中压力产生的加速度部分

$$\vec{a}_i^{Pressure} = -\frac{\nabla p(\vec{r}_i)}{\rho_i} = m \frac{45}{\pi h^6} \sum_{j=1}^N \left(\frac{p_i+p_j}{2\rho_i\rho_j} (h-r)^2 \frac{\vec{r}-\vec{r}_j}{r} \right) \quad (2-22)$$

粘度

现在计算由粘度产生的加速度

$$F^{Viscosity} = u \nabla^2 \vec{u}(\vec{r}_i) = u \sum_{j=1}^N \vec{u}_j \frac{m_j}{\rho_j} \nabla^2 W(\vec{r} - \vec{r}_j, h) \quad (2-23)$$

这个公式同样有不平衡的问题, 考虑到公式中的速度其实并不是绝对速度, 而是粒子间的相对速度, 所以这个公式的正确写法应该是:

$$F^{Viscosity} = u \sum_{j=1}^N m_j \frac{\vec{u}_j - \vec{u}_i}{\rho_j} \nabla^2 W(\vec{r} - \vec{r}_j, h) \quad (2-24)$$

其中的光滑核函数形式如下:

$$W_{Viscosity}(\vec{r}, h) = \begin{cases} K_{Viscosity}(-\frac{r^2}{2h^3} + \frac{r^2}{h^2} + \frac{h}{2r} - 1), & 0 \leq r \leq h \\ 0, & otherwise \end{cases} \quad (2-25)$$

$$\text{在 3D 情况下, } K_{Viscosity} = 15/(2\pi h^3) \quad (2-26)$$

由此可得到公式 2-12 的粘度部分

$$\vec{a}_i^{Viscosity} = \frac{F_i^{Viscosity}}{\rho_i} = mu \frac{45}{\pi h^6} \sum_{j=1}^N \left(\frac{\vec{u}_j - \vec{u}_i}{\rho_i \rho_j} (h - |\vec{r}_i - \vec{r}_j|) \right) \quad (2-27)$$

把公式 2-22 和 2-27 带入 2-12, 可以得到, 对于粒子 i , 它的加速度可以由公式 2-27 计算。

具体步骤如下:

- 1、初始化粒子状态, 为每个粒子赋上初始位置和速度信息
- 2、根据公式 2-16 计算每个粒子的密度
- 3、根据公式 2-19 计算每个粒子的压强
- 4、根据公式 2-27 计算每个粒子的加速度

- 5、根据临界条件调整加速度
- 6、根据加速度计算每个粒子的速度变化
- 7、根据速度计算粒子位置的变化
- 8、绘制粒子
- 9、回到步骤 2

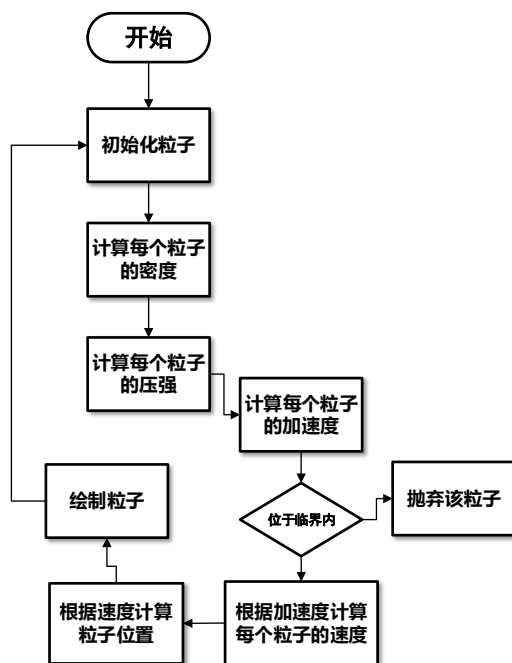


图 2-16 粒子更新流程图

2.5 本章小结

本章分四部分对基于粒子系统的海洋矢量场数据动态可视化方法进行描述，第一部分首先介绍了海洋环境数据、NefCDF 格式以及 NefCDF 的预处理方法，并将所需的矢量场数据进行解析为适合 Web 环境使用和传输的 JSON 格式。其次，介绍了针对常见海洋矢量场数据可视化的五种方法，并进行比较分析。然后介绍了 WebGL 技术下 SPH 实时流体模拟方法的具体实现过程与粒子搜索算法的改进，并进行了实验结果的对比分析。最后主要介绍了基于 SPH 进行海洋矢量场数据动态可视化的算法的关键点，并给出其中重要的公式推导，最后给出整个算法实现的技术流程图。

其中，在第三部分对 WebGL 技术下的 SPH 实时流体模拟方法的阐述中，首次提出了一种基于多帧缓冲区的 SPH 实时流体模拟方法具体框架，分为初始的初始化，更新和渲染的过程，对原有 SPH 整个算法的具体实现，针对 WebGL 技术的特点，在 GPU 中用多个纹理缓冲区来存储粒子位置以及相关的中间信息，

从而减少了 CPU 和 GPU 之间的数据传输。通过改进后的邻域粒子搜索算法快速找出相邻粒子，将多个中间计算结果送入离屏帧缓冲区进行融合，最后将所有离屏帧缓冲区中的内容送入到主帧缓冲区，在 GPU 中直接进行绘制。使得在进行实时流体模拟时，具有很好的实时仿真渲染帧率，能够满足在 Web 前端进行三维场景渲染的帧率要求。

3. 大规模矢量场数据可视化场景的渲染加速方法

在 Web 前端进行大规模矢量场实时渲染时, 需要实时更新粒子系统中每个粒子的状态, 并实时计算其下一时刻的状态, 这对浏览器有很高的计算量要求, 对系统的资源开销也非常大, 在实时计算量较大的三维 Web 应用中, 如何平衡 CPU 和 GPU 以及平衡执行单线程中计算和渲染资源分配是一个比较大的问题。为了通过对 Web 应用的优化减小系统资源的开销, 实现 CPU 与 GPU 的负载平衡, 以下分别使用 WebCL, WebWorker 多线程, 基于 V8 引擎的分级垃圾回收算法, 多帧缓冲区的 GPU 加速算法对整个可视化场景进行渲染加速。

3.1 WebCL 并行计算

WebCL (Web Computing Language) 是 Khronos Group 团队于 2014 年 3 月正式发布的一款标准。该标准为 Web 引入了新的 DOM 对象 `webcl`, 并为该对象提供了一系列的程序调用接口。开发者通过操作 DOM 对象 `webcl`, 以及对于 `webcl` 对象接口的合理调用, 为 Web 程序提供并行执行的能力。Web 应用通过 WebCL 提供的并行能力加速应用中耗费大量计算力的部分, 从而提高整个 Web 应用的性能。WebCL 项目使用 JavaScript 在 web 上实现 OpenCL, 允许 web 开发者可以直接接触到 GPU 和多核 CPU 这些比较大规模的并行程序设计。WebCL 底层依赖于 OpenCL 实现, 因此其运行模型与 OpenCL 相似。而 OpenCL 是一个为异构平台编写程序的框架, 此异构平台可由 CPU, GPU 或其他类型的处理器组成。OpenCL 由一门用于编写 kernels (在 OpenCL 设备上运行的函数) 的语言和一组用于定义并控制平台的 API 组成。OpenCL 提供了基于任务分割和数据分割的并行计算机制。)(Leonwei, 2013)。

WebCL 运行模型如图 3-1 所示。使用 WebCL 的程序需要建立相应的程序运行环境。首先, 获取相应的硬件支持信息并完成 WebCL 基本对象的构造。通过 WebCL 提供的 `platform` 信息查询接口, 并获得程序运行的平台上支持 WebCL 的设备信息。之后可以根据该设备创建 相应的 WebCL 上下文运行环境。

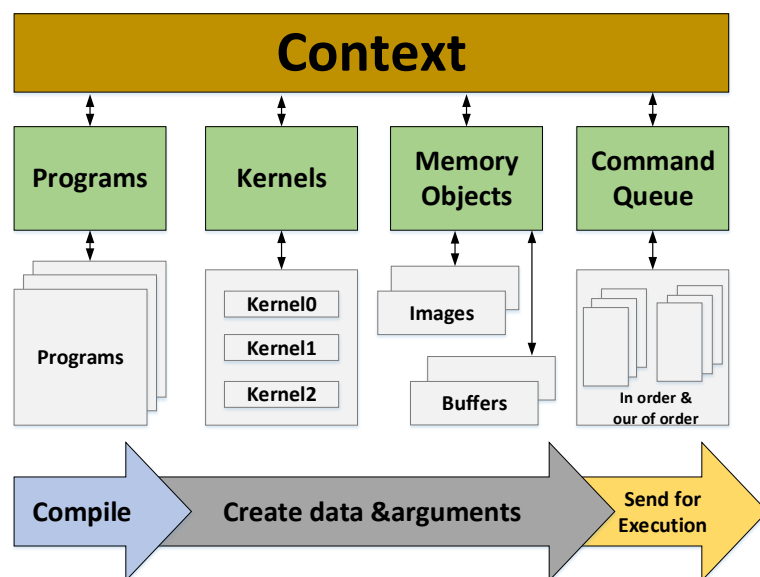


图 3-1 WebCL 运行模型

同一台设备可以创建多个 WebCL 上下文运行环境。同时，WebCL 的所有对象都是基于上下文的，不同上下文所依赖的 WebCL 对象也不同，且大多无法跨上下文交互。创建上下文后，基于上下文和 cl 文件创建 WebCL 在该上下文环境中的程序对象，并根据该对象创建内核函数对象。同时，需要创建 WebCL 的命令队列对象，该对象对于 WebCL 命令的执行十分重要。所有的 WebCL 的指令执行，都是将指令加入队列，等待返回的过程。这种模型十分适合基于事件的异步执行方式。完成上述对象构建后，便完成了 WebCL 的内核函数运行的准备工作，调用内核函数的相关方法向内核函数传入参数值。同时，此时也可以向命令队列中压入执行命令，内存读写命令等，完成内核函数的执行与结果输出 (Munshi, 2012)。

3.1.1 WebCL 实现

本研究在 Cesium 基础上构建 Web 数字地球系统，而 Cesium 是基于 WebGL 底层实现，在更新粒子系统中所有粒子的状态时，需要在顶点着色器和片元着色器中对粒子状态进行计算。这是，我们将其中的复杂计算交给 WebCL，将会大大提高粒子状态更新的速度，其中的办法就是创建 WebCL 与 WebGL 共享内存对象 (闫绍博, 2015)，其实现流程如图 3-2 所示，具体步骤如下。创建 WebCL 与 WebGL 共享内存对象的前提条件是在设备上启动 KHR_gl_sharing 扩展协议。步骤如下：

(1) Web 应用开发者调用 JavaScript 中 createContext 界面并传入 WebGL 上下文与设备对象作为函数参数

(2) Google Chrome V8 引擎解析 createContext 接口并传入参数，调用 Blink

渲染引擎中的绑定模块。

- (3) 绑定模块确定调用 WebCL 中的 WebCL 对象。根据传入参数进行分析，调用 WebCL 对象中接受 WebGL 上下文以及设备对象的 createContext 函数。
- (4) WebCL 对象检查 WebGL 上下文对象的有效性。
- (5) WebCL 对象检查运行设备对象的有效性。
- (6) WebCL 对象调用 OpenCL 中 clCreateContext 接口。
- (7) OpenCL 完成创建上下文对象操作后返回上下文对象。
- (8) 调用 WebCLContext 的 create 函数完成 WebCL 上下文对象的初始化。

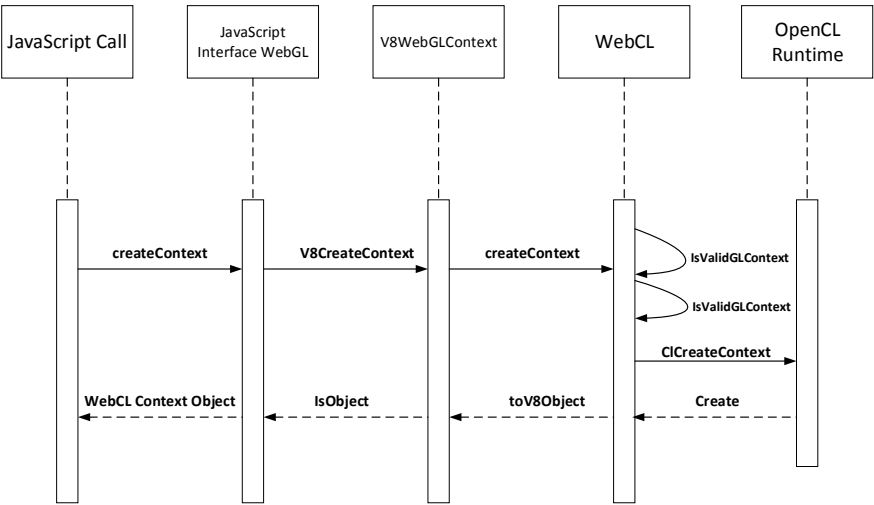


图 3-2 创建 WebCL 与 WebGL 共享内存对象

本文中并行算法主要用于计算粒子系统中粒子对搜索半径内粒子的搜索并进行一次粒子系统更新，在硬件设施为 CPU Intel Xeon，内存 8.00GB，GPU NVIDIA Quadro FX 5800；软件环境为 Window 7 X64，Google Chrome 浏览器，在“全球环境资源空间信息系统”的测试平台上随机生成不同粒子数的粒子系统测试结果如表 3-1 所示。由该结果可以得出 WebCL 并行计算的效率约为串行计算效率的 3 倍。

表 3-1 并行计算与串行计算时间对比表（单位：ms）

粒子数(10 万)	串行计算	并行计算
1	751.41635	251.47731
2	837.35973	279.21273
3	917.31576	305.77631
4	1074.48583	357.21374

3.2 WebWorker 多线程

至 2008 年 W3C 制定出第一个 HTML5 草案开始,HTML5 承载了越来越多崭新的特性和功能。它不但强化了 Web 系统或网页的表现性能,而且还增加了对本地数据库等 Web 应用功能的支持。其中,最重要的一个便是对多线程的支持。在 HTML5 中提出了工作线程(Web Worker)的概念,并且归纳出 Web Worker 的三大主要特征:能够长时间运行(响应),理想的启动性能以及理想的内存消耗。

在 HTML5 中,工作线程的出现使得在 Web 页面中进行多线程程序设计成为可能。Web Worker 的基本原理就是在当前 JavaScript 的主线程中,使用 Worker 类加载一个 JavaScript 文件来开辟一个新的线程,起到互不阻塞执行的效果,并且提供主线程和新线程之间数据交换的接口: `postMessage`, `onmessage`。众所周知,传统页面中(HTML5 之前)的 JavaScript 的运行都是以单线程的方式工作的,如图(浏览器构成的图示)为浏览器主要构成,当我们请求一个网页时,浏览器主要有两个部分在交替工作,渲染引擎和 JS 解释器,为了保证网页接口的一致性,浏览器选择了单线程的工作方式。单线程在之前较为简单的网页来说是可行的,随着近些年 Web 开发技术的发展,Web 前端执行的任务越来越多,也越来越复杂,就催生出了 WebWorker 多线程的工作方式。目前,虽然有多种方式实现了对多线程的模拟(例如:JavaScript 中的 `setInterval` 方法,`setTimeout` 方法等),但是在本质上程序的运行仍然是由 JavaScript 引擎以单线程调度的方式进行的。

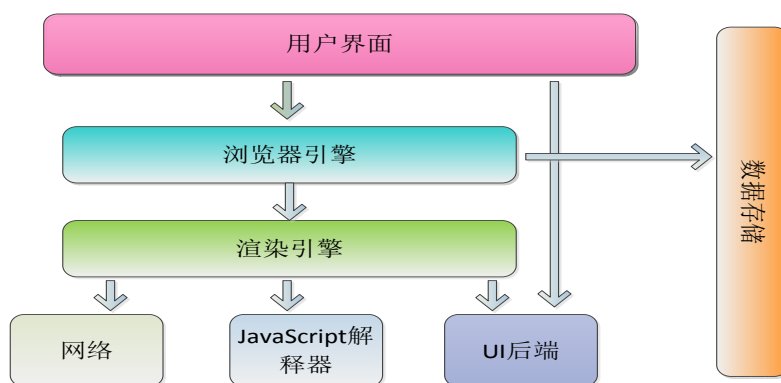


图 3-3 浏览器主要组件构成图

传统上的线程可以解释为轻量级进程,它和进程一样拥有独立的执行控制,一般情况下由操作系统负责调度。而在 HTML5 中的多线程是这样一种机制,它允许在 Web 程序中并发执行多个 JavaScript 脚本,每个脚本执行流都被称为一个线程,彼此间互相独立,并且有浏览器中的 JavaScript 引擎负责管理,如图 3-3。本研究拟采用 WebWorker 多线程的方法,将矢量场动态可视化的任务中的

实时逻辑计算部分分离，并交给开辟的 WebWorker 多线程进行处理，同时实时地将计算结果传递给浏览器的渲染引擎进行渲染，这样可以利用多线程的优点将复杂的任务分解处理，最后达到矢量场流畅的可视化效果和交互操作。

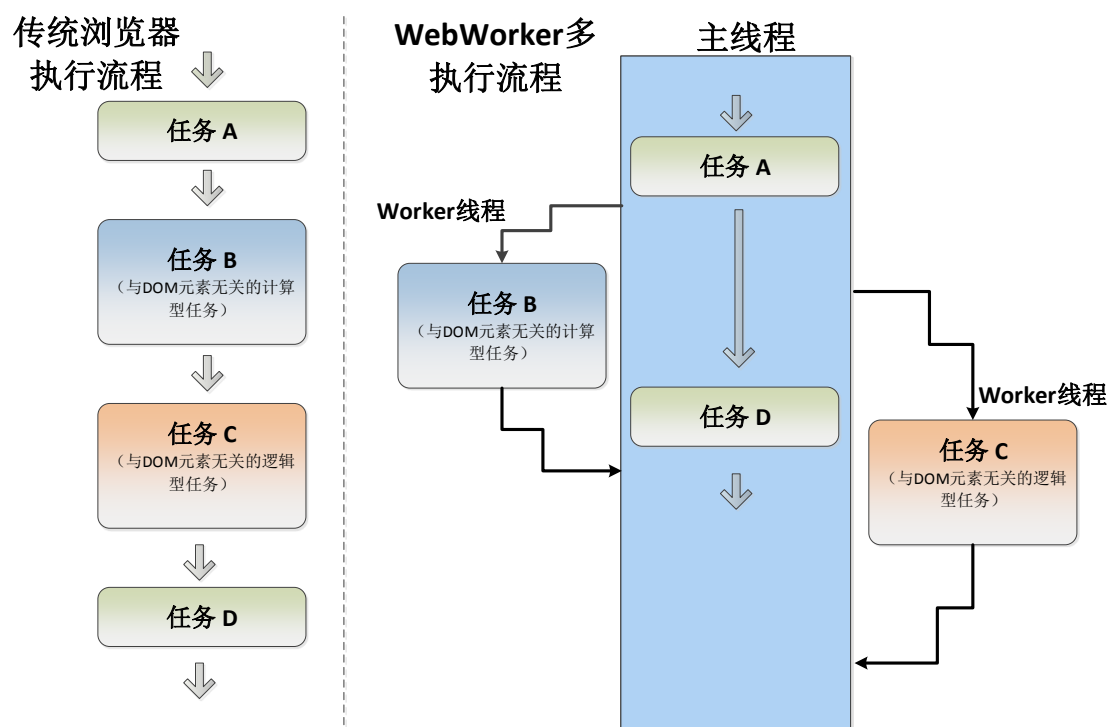


图 3-4 浏览器传统工作方式与 WebWorker 多线程方式对比示意图

HTML5 引入了一个工作线程（webWorker）的概念，它允许开发人员编写能够长时间运行而不被用户所中断的后台程序，去执行事务或者逻辑型计算任务，处理一些比较耗时的计算，同时保证页面对用户的及时响应。这样就允许 JavaScript 创建多个线程，但是子线程完全受主线程控制，且不得操作 DOM。下面是一个简单的示例：

```
//创建一个 Worker 对象，并向它传递将在新线程中执行的脚本 url
var worker = new Worker('worker.js');
worker.postMessage(value);
//接收 worker 传递过来的数据
worker.onmessage = function(event){
    //do something
};
```

下面是 worker.js 的内容：

```
//do something
//将得到的 value 发送回主线程
postMessage(value);
```

通过以上对 WebWorker 原理与应用的阐述,本文中主要将 WebWorker 应用于更新粒子系统时计算当前粒子的相关属性信息,如加速度,位置。因此,本文针对由 4 万粒子组成的粒子系统对海洋矢量场数据进行动态可视化更新,分别为不使用 WebWorker,开辟一个到 10 个 WebWorker,针对相同粒子系统进行计算,统计时间对比表见表 3-2。该表结果表明随着 WebWorker 个数的增加,计算时间越短,其中仅开两个 WebWorker 时,效率提高了 8 倍,但是,并不是越多的 WebWorker 计算时间就越快,是一个 U 形的曲线,总体来说,使用 WebWorker 是优于不使用 WebWorker 的计算效率的。

表 3-2 WebWorker 个数与计算时间对比表(单位: s)

WebWorker 个数	多线程	不采用多线程
1	0.514	1.627
2	0.294	1.627
4	0.195	1.627
6	0.168	1.627
10	0.192	1.627

当然,WebWorker 也有其本身的局限性,比如不能跨域加载 JS 和 worker 内代码不能访问 DOM 等,只有在使用该技术时,巧妙的设计应用程序计算逻辑,干净利落的拆分出纯粹的重复且复杂的任务,交给 WebWorker 多线程进行执行,这样既不影响主页面的响应,且快速地执行应用程序,拥有较好的用户体验。

3.3 基于 V8 引擎的分级内存回收算法

在使用粒子系统可视化矢量场的过程中,会不断有新的粒子产生、到达生命周期尽头的粒子消亡和优势速度粒子的保留或增强,那么我们就需要一种好的管理方式来分别对这些粒子进行管理。本研究借鉴 Google Chrome 浏览器中 V8 引擎使用的分代内存管理方法,该方法针对垃圾回收分配两块内存,分别为新生代和老生代。本研究针对初试速度不同的新生粒子粗略将粒子分配到不同的两个区块中,然后在其生命周期内动态的进行粒子回收和新的粒子的分配。

新生代使用 Scavenge 算法进行回收。在 Scavenge 算法的实现中,主要采用了 Cheney 算法。Cheney 算法是一种采用复制的方式实现的垃圾回收算法。它将内存一分为二,每一部分空间称为 semispace。在这两个 semispace 中,一个处于使用状态,另一个处于闲置状态。处于使用状态的 semispace 空间称为 From 空

间，处于闲置状态的空间称为 To 空间，当我们分配对象时，先是在 From 空间中进行分配。当开始进行垃圾回收算法时，会检查 From 空间中的存活对象，这些存活对象将会被复制到 To 空间中（复制完成后会进行紧缩），而非活跃对象占用的空间将会被释放。完成复制后，From 空间和 To 空间的角色发生对换。也就是说，在垃圾回收的过程中，就是通过将存活对象在两个 semispace 之间进行复制。可以很容易看出来，使用 Cheney 算法时，总有一半的内存是空的。但是由于新生代很小，所以浪费的内存空间并不大。而且由于新生代中的对象绝大部分都是非活跃对象，即等待回收的对象，需要复制的活跃对象比例很小，所以其时间效率十分理想。复制的过程采用的是 BFS（广度优先遍历）的思想，从根对象出发，广度优先遍历所有能到达的对象。新生代执行过程大致如下图所示：

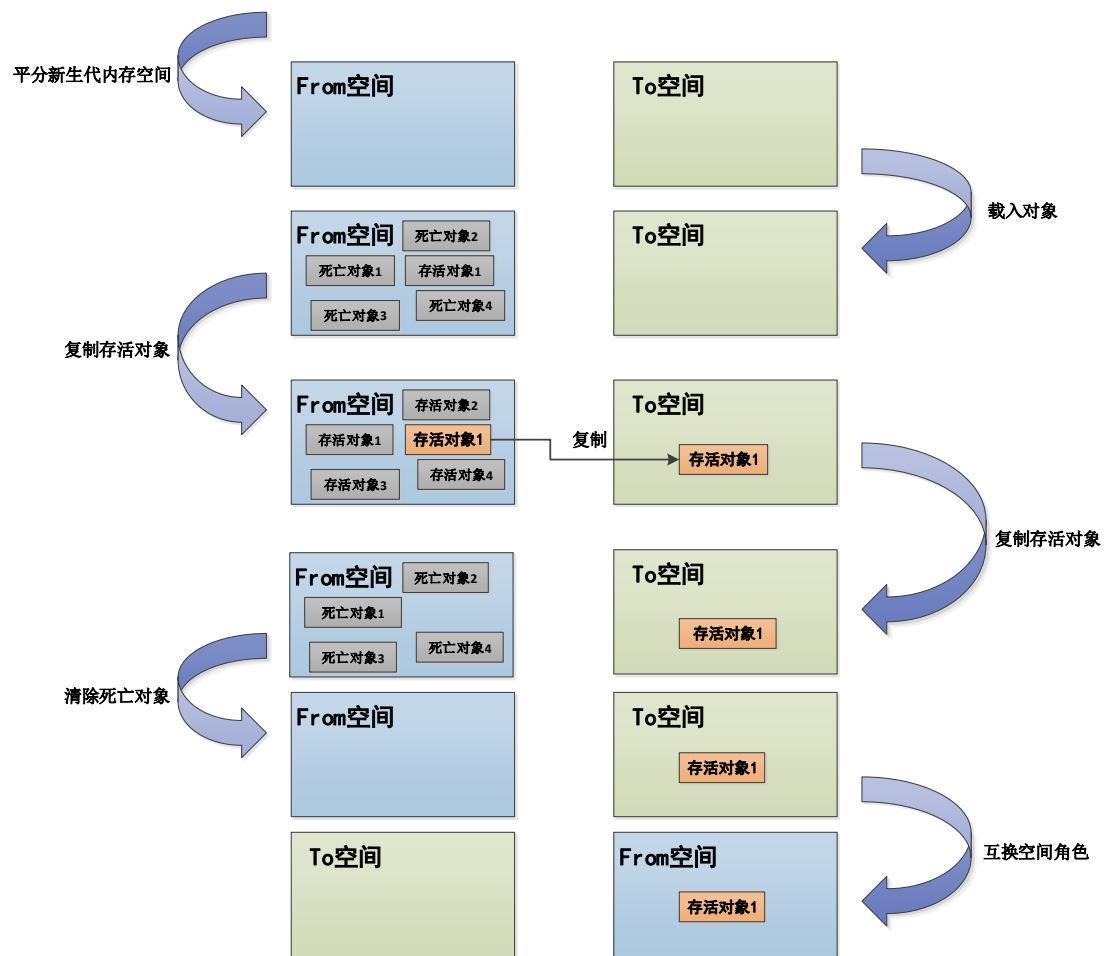


图 3-5 新生代垃圾收集过程图

当一个对象经过多次新生代的清理依旧幸存，这说明它的生存周期较长，也就会被移动到老年代，这称为对象的晋升。老年代所保存的对象大多数是生存周期很长的甚至是常驻内存的对象，而且老年代占用的内存较多。老年代占用内存较多（64 位为 1.4GB，32 位为 700MB），如果使用 Scavenge 算法，浪费一半空间不说，复制如此大块的内存消耗时间将会相当长。所以 Scavenge 算法显然不

适合。V8 在老生代中的垃圾回收策略采用 Mark-Sweep 和 Mark-Compact 相结合。Mark-Sweep（标记清除）分为标记和清除两个阶段。在标记阶段需要遍历堆中的所有对象，并标记那些活着的对象，然后进入清除阶段。在清除阶段时，只清除没有被标记的对象。由于标记清除只清除死亡对象，而死亡对象在老生代中占用的比例很小，所以效率较高。标记清除有一个问题就是进行一次标记清除后，内存空间往往是不连续的，会出现很多的内存碎片。如果后续分配一个需要内存空间较多的对象时，如果所有的内存碎片都不够用，将会使得 V8 无法完成这次分配，提前触发垃圾回收。

Mark-Compact（标记整理）正是为了解决标记清除所带来的内存碎片的问题。标记整理在标记清除的基础进行修改，将其的清除阶段变为紧缩极端。在整理的过程中，将活着的对象向内存区的一段移动，移动完成后直接清理掉边界外的内存。紧缩过程涉及对象的移动，所以效率并不是太好，但是能保证不会生成内存碎片。老生代使用标记清除和标记整理结合的方式，主要采用标记清除算法，如果空间不足以分配从新生代晋升过来的对象时，才使用标记整理。其具体工作方式如下图所示。

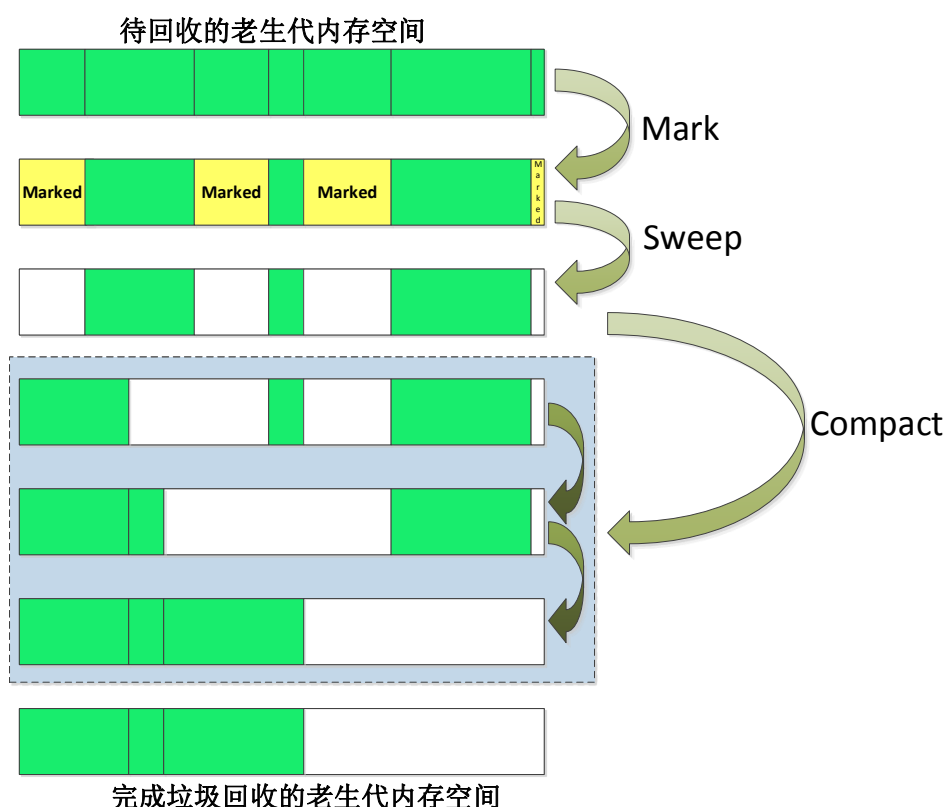


图 3-6 老生代内存空间回收流程图

3.4 多帧缓冲区的 GPU 加速算法

利用多个帧缓冲区的方法，将粒子系统更新计算的中间数据全部储存在帧缓存中，方便读取，然后将计算结果写入到一块独立的帧缓冲区，最后将此结果缓冲区内容写入到主帧缓冲区，一次性渲染，这大大节省了渲染的时间，同时以上过程全部在 GPU 中执行，大大减少了 CPU 和 GPU 之间的数据交换和解析工作，这都对整个粒子系统的渲染帧率有了很大的提升，算法具体原理如图 3-7。

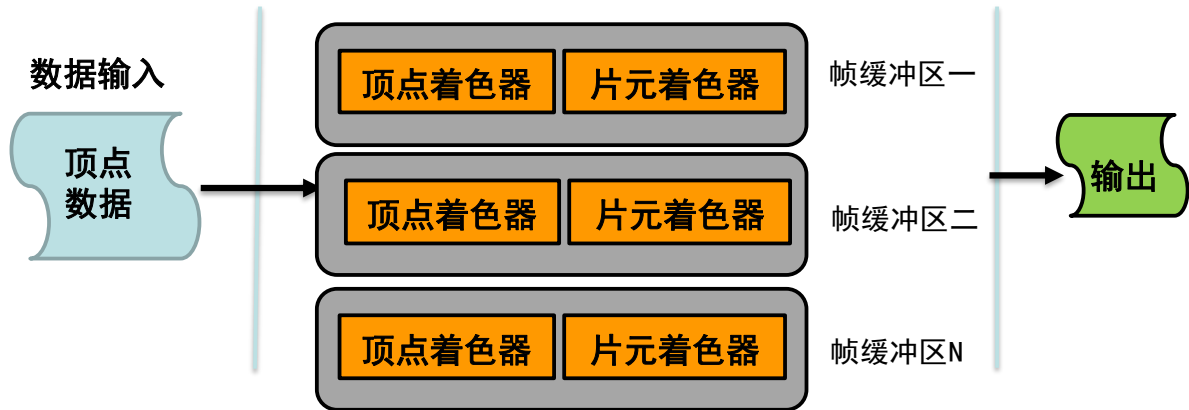


图 3-6 老生代内存空间回收流程图

使用离屏渲染（OffScreen Rendering）技术，在 GPU 在当前屏幕缓冲区以外新开辟一个帧缓冲区进行渲染操作。根据之前多次计算的结果，将所有需要绘制的数据结果存储到一个帧缓冲区，然后一次性绘制，减少了多次渲染所耗费的计算资源。本文提出的这个方法，可以有效解决本节开始提到的几个问题，将大部分计算全部放在 GPU 中执行，这在 Web 三维应用程序苛刻的运行环境下，如单线程和数据传输耗时等，大大减轻了 Web 前端程序的计算任务负担。同时中间的计算结果全部在 GPU 中存取，大大减少了 CPU 和 GPU 的数据交换带来的损耗，减轻了 CPU 的负担，能够有效提高渲染的帧率。

为了进一步证明多帧缓冲区 GPU 加速所达到的效果，分别用 CPU 结合 GPU 以及 GPU 渲染不同个数的粒子，且粒子个数逐渐增多，实验结果如表 4-5 所示，该结果说明随着粒子个数的逐渐增多，GPU 的渲染效率是 CPU 结合 GPU 渲染效率的将近 2 倍。

表 3-3 CPU 与 GPU 渲染效率对比表（单位：FPS）

粒子个数（万）	GPU	CPU+GPU
1	58	60
4	32	38
10	21	15
16	16	9
20	14	5

3.5 本章小结

本章从 4 个方面介绍了论文对 WebGL 场景中大规模实时计算和渲染进行优化加速的方法，即 WebCL，WebWorker 多线程，基于 V8 引擎的分级内存回收算法以及多帧缓冲区的 GPU 加速算法。其中，WebCL 使得 Web 应用程序能够合理调用多核 CPU 的资源，Web 应用程序中偏重于逻辑型计算的任务，论文使用 WebWorker 多线程模型对任务进行分离。同时，论文介绍了基于 V8 引擎的分级内存回收算法，详细阐述了 Web 应用程序运行中出现的新生代和老生代内存的交换和回收的方法，并予以实现。最后，针对，WebGL 大规模场景的实时渲染，本文利用多个帧缓冲区的方法，使得实时计算的中间过程全部在 GPU 中执行，大大减少了 CPU 和 GPU 之间的数据交换和解析工作，对整个海洋矢量场数据的动态渲染效率有了较大的提升。

4. 海洋矢量场数据动态可视化的设计与实现

上述章节介绍了基于粒子系统的海洋矢量场数据动态可视化的方法以及多种大规模可视化场景的渲染加速方法,在本章中,将要借助科学的软件设计理论和计算机编程的形式将方法具体应用到实际的物理场景中,包括构造基于 Cesium 的 Web 数字地球系统,基于粒子系统的海洋矢量场数据的动态可视化,并将实现的结果进行对比分析。

4.1 基于 Cesium 的 Web 数字地球原型系统构建方法与实现

大规模海洋矢量场的动态可视化需要在大尺度的范围上展示才能查看其总体的运行趋势,基于此,本文基于 Cesium 引擎构建 Web 数字地球系统,Cesium 是一套用来渲染 3D 地球,2D 区域地图,和多种 GIS 要素的 JavaScript 库,其能够进行地理空间数据的三维可视化、具有非常高的性能和精度,并且 Cesium 能够方便的切换二维、2.5 维和三维球体模式。本文拟从自定义 shader 的多通道渲染,基于 Primitives 的复杂几何体渲染方法和 Requirejs 模块化代码组织放了三个方法来对 Web 数字地球系统构建研究。

4.1.1 自定义 shader 的多通道渲染

Cesium 封装了 WebGL 复杂的底层实现,这样开发者可以抛弃复杂的 WebGL 原生接口,而不用考虑顶点着色器、片元着色器以及纹理映像等问题。Cesium 的体系架构按层级划分,且主要包括核心层、渲染器层、场景层,数据源层和小部件层五个部分,其中渲染器层对 WebGL 所提供的功能进行了复杂的封装与抽象,因为应用开发人员不用接触底层复杂的 WebGL 代码逻辑,该层提供的功能比直接使用 WebGL 功能更加简洁,语义更加明确 (Cozzi, 2015)。渲染器层内置了 GLSL 常用相关渲染功能、顶点着色器和片元着色器的抽象表示、纹理、顶点缓冲区及渲染状态等,其整体结构如下:

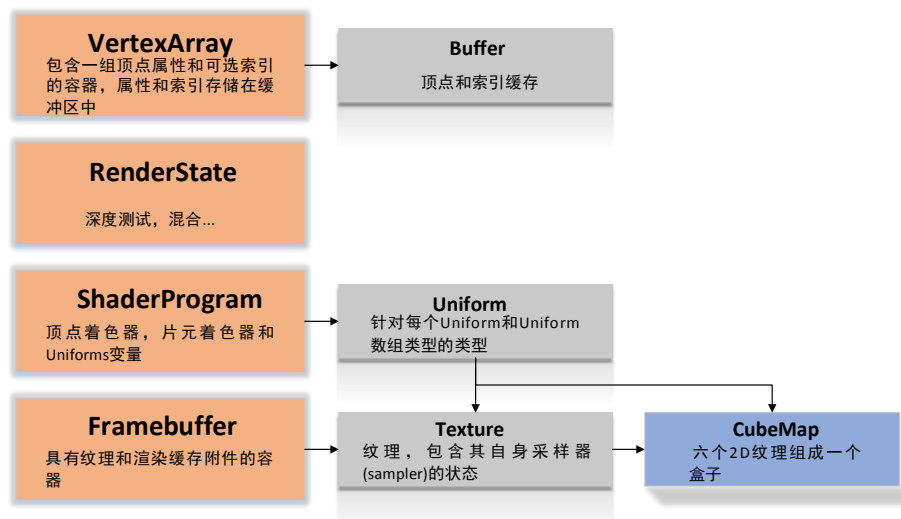


图 4-1 渲染器（Renderer）组成示意图

上图左侧的四个渲染元件：**VertexArray**，**RenderState**，**ShaderProgram** 和 **Framebuffer**：构成了 Cesium 中绘制命令（DrawCommand）的基础，每一个绘制命令都封装了部分场景的绘制请求。基于 Cesium 的应用每渲染一帧都会在绘制资源池中调用很多绘制命令进行绘制。

VertexArray：具有一组顶点属性和可选索引的容器，属性和索引存储在缓冲区中，顶点数组使用了 WebGL 中 OES_vertex_array_object 扩展，这样可以减少 WebGL 的调用次数。

RenderState：包含发布绘制请求所需的 WebGL 固定功能状态，比如深度，混合和模板状态。这是 WebGL 细粒度状态的粗粒度表示，实际上映射到现在的 GPU，如果一个类似 Vulkan API（跨平台的 2D 和 3D 绘图应用程序编程接口）在 Web 上展现，它应该很容易被更新。

ShaderProgram：包含了一个用来编译和链接的 WebGL Program 对象和 Uniforms 变量，Uniforms 变量可以直接使用 Cesium 中的矩阵，笛卡尔坐标，颜色，纹理和立方体地图类型。

Framebuffer：具有纹理和渲染缓存的容器，并且它是绘制请求的目标。

常见的解决方案的思路是在 Cesium 中分散 WebGL 的调用次数和频率，但是将这些 WebGL 调用集中在渲染器（Renderer）中会具有很多优点：

- ◆ 易于使用，渲染器提供了比 WebGL 更高级的抽象，因此这会使 Cesium 余下的部分更加简洁且不易出错。
- ◆ 着色器管道（Shader Pipeline），Cesium 的使用需要一个运行时着色器管道，用来生成着色器以及拥有 Uniforms 变量和函数的 GLSL 库，这也是渲染器的一部分。
- ◆ 性能：优化和最佳实践，缓存和最小化 WebGL 调用被应用在一个地方，而

其余的 Cesium 都会从这里受益。

- ◆ 状态(State): WebGL 是一个状态机, 并且渲染器在一个地方统一管理状态。
- ◆ 便携性: 渲染器可以方便地添加 WebGL 扩展, 升级到 WebGL 2, 并且为特定平台添加替代解决方法。

着色器管线

着色器是使用独立的 .glsl 文件制作并广泛使用的。Cesium 在构建整个 JavaScript 库文件时, 会删除 GLSL 的注释和空格, 并将 GLSL 文件转换为一个 RequireJS 模块, 这些 RequireJS 模块以一个 JavaScript 字符串的形式返回, 这样剩余的 Cesium 文件就可以方便的引用它们, 而不需要对每个文件都进行请求。GLSL 着色器的源代码位于 Source/Shaders 目录中。

由于 Cesium 中的很多着色器是完全或者部分手写, 所以 Cesium 提供了一个包含函数(function), Uniforms 变量(uniform), 结构(struct)和常量(constant)的大型 GLSL 库, 我们可以在不声明变量或者不使用常见的#include 标记的情况下使用这个 GLSL 库。这些标识符以 czm_开头。例如, 下面的代码是渲染天空颜色的片元着色器的片段:

```
czm_ellipsoid ellipsoid = czm_getWgs84EllipsoidEC();
vec3 direction = normalize(v_positionEC);
czm_ray ray = czm_ray(vec3(0.0), direction);
czm_raySegment intersection = czm_rayEllipsoidIntersectionInterval(ray,
ellipsoid);
if (!czm_isEmpty(intersection)) {
    discard;
}
```

GLSL 内置函数可以引用其他的内置函数, 进而形成定向非循环图(DAG)。在运行时, GLSL 源文件被提供给 ShaderSource 变量, ShaderSource 找到 czm_标识符, 然后沿着定向非循环图(DAG)生成最终的 ShaderSource。如果着色器将被用于拾取, 那么它也会被打包进而输出一个选择 ID, 而不是实际的颜色。上面的过程是在运行时完成的, 而不是在 Cesium 库文件构建时完成的, 这样可以避免下载同一内置 GLSL 的几个副本, 并且因为一个 Cesium 应用程序可能需要很多排列的着色器, 这些着色器直到运行时才会被确定。

着色器程序使用原始的 GLSL 源文件时被缓存, 这样可以作为减少 WebGL 调用次数的关键, 而 WebGL 的调用都是由初始化和使用着色器组成。

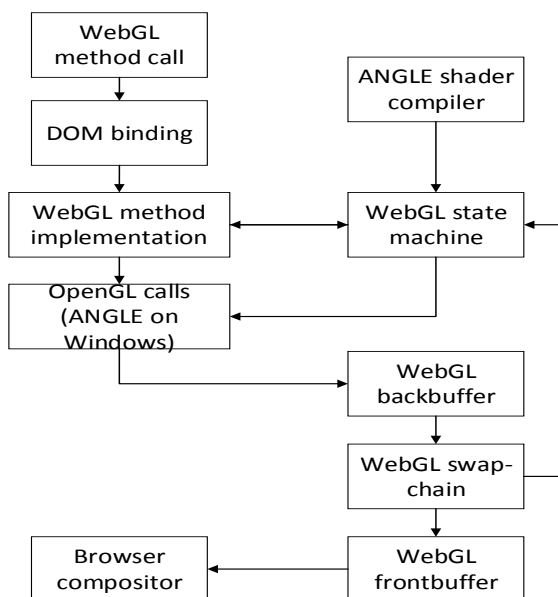


图 4-2 WebGL 在浏览器中执行流程

4.1.2 基于 Primitives 的复杂几何体渲染方法

Cesium 与一般的图形渲染引擎类似，但随着我们提升 Cesium 抽象的层次，其类别归属就更具体到 Cesium 的目标领域：虚拟地球(Cozzi, 2015)。Cesium 的图形渲染栈中最低层次的是渲染器层，它是一个处理 WebGL 资源管理和绘制命令的 WebGL 抽象层。一个绘制命令（Draw Command）表示 WebGL 绘制请求和执行它所需的状态变量，如着色器，Uniforms 变量，顶点数组等等。场景层是构建与渲染器层之上的，而构建在渲染器和场景层上的是 Cesium 图形渲染栈的最顶层，就是基元（Primitives），它们通过创建绘制命令并将其提供给场景层来呈现真实世界对象。

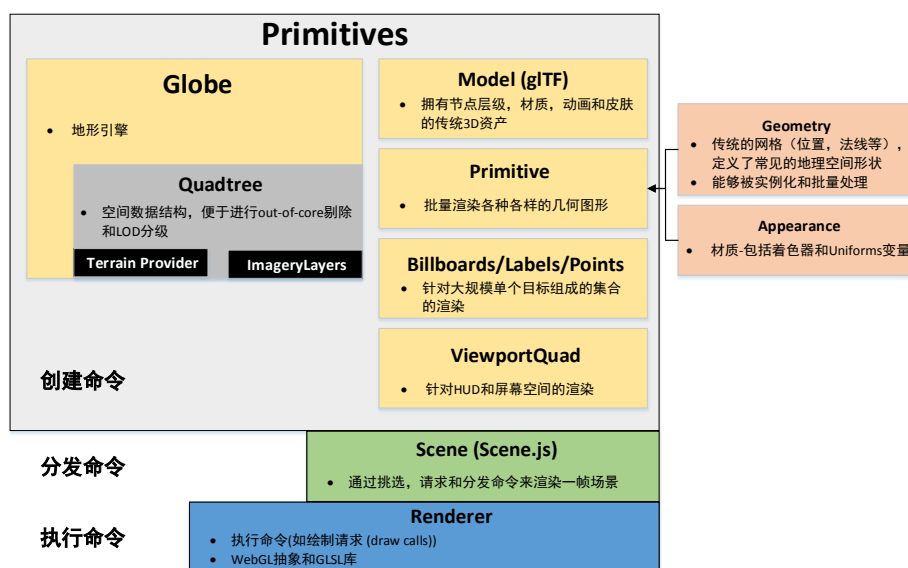


图 4-3 Cesium 图形渲染栈

1、Globe

Globe 基元渲染了地球球体, 包括地形, 影像图层和动态的海洋表面。Ring 在其文章中描述(Ring, 2013), Cesium 在地理坐标中使用四叉树用来表示 out-of-core 的层级 LOD (HLOD)。QuadtreePrimitive 目前尚未成为 Cesium API 的一部分, 所以如果使用 QuadtreePrimitive, 可能在地形的显示上会有断层。地形切片包含了几何图形, 在一个给定的视角, 一组地形切片就能近似模拟地球的表面。

当场景层调用 Globe 的更新功能时, 四叉树就会被遍历并返回地形切片的命令。每个命令都会为每个重叠的影像切片引用纹理, 并使用基于重叠的影像切片数量和其他条件(如启用光照, 后处理过滤或者动态的水的效果)的依照程序而生成的着色器。着色器是使用单一大型着色器(uber shader)和字符串连接生成的。

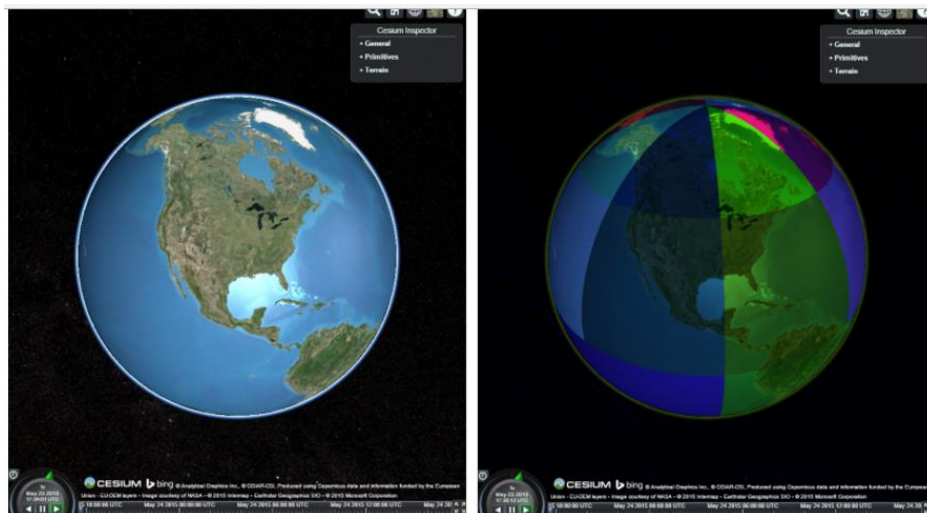


图 4-4 Globe 默认视图和开启调试视图

3、Primitive

Cesium 的一般基元简称为 Primitive, 这是 Cesium 和一般游戏引擎不同的一个区域。游戏引擎通常包含一些最小化的几何, 如折线, 球体和多边形, 可用于一些边界情况, 如调试和平视显示(HUD)。相反, 游戏引擎侧重于环境, 资产和效果, 如光照, 粒子系统和后处理。

另一方面, Cesium 包含了用于在地球上绘制常见物体的大型几何图形集, 例如折线, 多边形和挤压立方体。大多数的 Cesium 应用程序, 比如下图的 Air Tasking Orders Visualizer, 使用几何图形来绘制大部分的内容。这与游戏完全不同, 游戏中的大部分内容是 3D 模型财产。

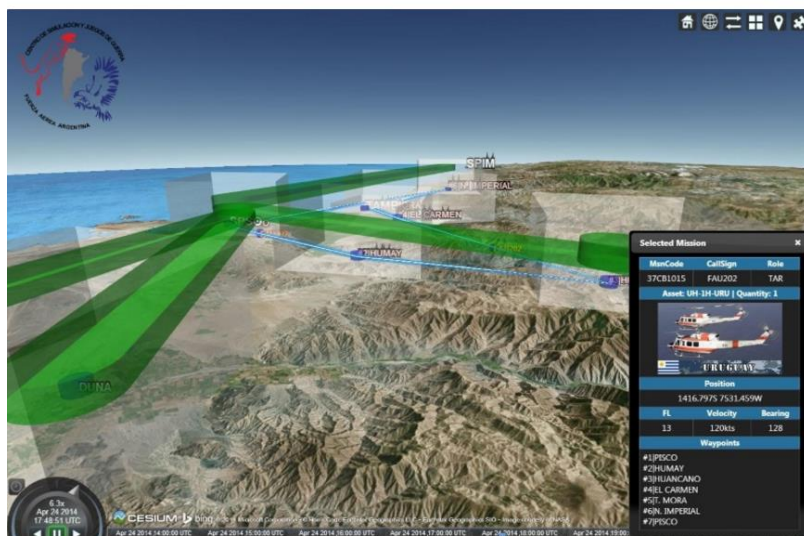


图 4-5 WebGL 应用 ATO Visualizer (Wargaming and Simulation Center of the Argentinian Air Force)

最终的着色器是通过修改外观的顶点和片段着色器生成的，这样做是为了利用相对于眼镜 (GPU RTE) (Ohlarik, 2008) 技术来达到高精度渲染，对 3D/2D/Columbus-view 支持，单个几何体的显示/隐藏和单元向量压缩(Bagnell, 2015)。同时，用来批量处理几何体的几何体管线对顶点属性进行相应的修改，例如分离用于 GPU RTE 的低/高位置字节，正常的压缩等。

4、Billboards/Labels/Points

在游戏引擎中，布告板 (Billboard) 通常用于烟雾和火灾等离子系统中。Cesium 的 Billboard 用于更通用的目的，通常用来代表移动的物体，如图 4-6 的卫星实时追踪和 Doarama 的滑翔伞。

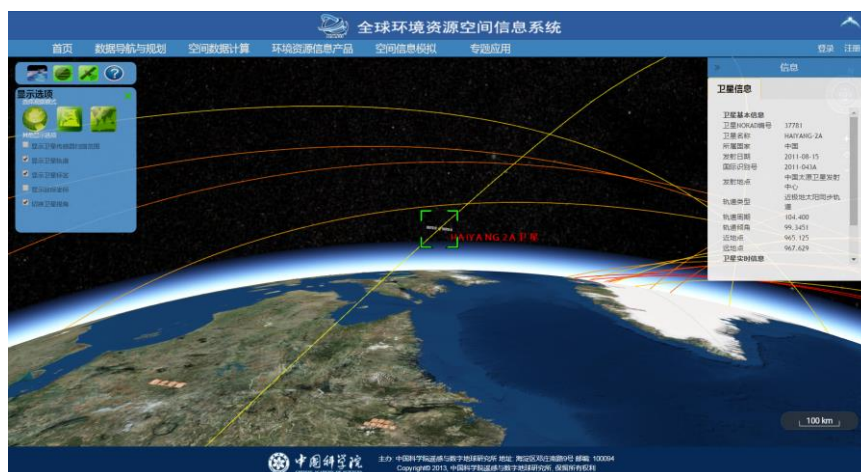


图 4-6 全球环境资源空间信息系统-卫星轨道模拟 (<http://gers.radi.ac.cn>)

Cesium 的布告板提供了很多灵活性，例如显示/隐藏，设置像素和视野偏移，设置屏幕空间旋转，并且可以根据视野距离来调整缩放尺寸。这会导致“胖顶点” (fat vertices)，所以 Cesium 使用打包和压缩来最小化顶点属性的数量。单一大

型着色器（uber shader）使用 GLSL 预处理器来关闭不必要的功能，例如，如果没有布告板被旋转，则顶点着色器不会对旋转进行任何数学运算。这在 Cesium 中是非常常见的，与大多数游戏相反，这些游戏则具有复杂的片元着色器和简单的顶点着色器。

4.1.3 Requirejs 模块化代码组织方式

AMD 是 "Asynchronous Module Definition" 的缩写，意思就是 "异步模块定义"。它采用异步方式加载模块，模块的加载不影响它后面语句的运行。所有依赖这个模块的语句，都定义在一个回调函数中，等到模块加载完成后，这个回调函数才会运行(阮一峰，2012)。

AMD 也采用 `require()` 语句加载模块，但是不同于 CommonJS，Requirejs 模块加载如图 4-8，它要求两个参数：

`require([module], callback);`

第一个参数 `[module]`，是一个数组，里面的成员就是要加载的模块；第二个参数 `callback`，则是加载成功之后的回调函数。如果我们把常见代码写成 AMD 形式，如下面这样：

```
require(['Module'], function (args) {  
    args.add(2, 3);  
});
```

`args.add()` 与 `Module` 模块加载不是同步的，浏览器不会堵塞，甚至发生假死。很显然，AMD 比较适合浏览器的前端运行环境。

目前，主要有两个 JavaScript 库实现了 AMD 规范：Require.js 和 curl.js。

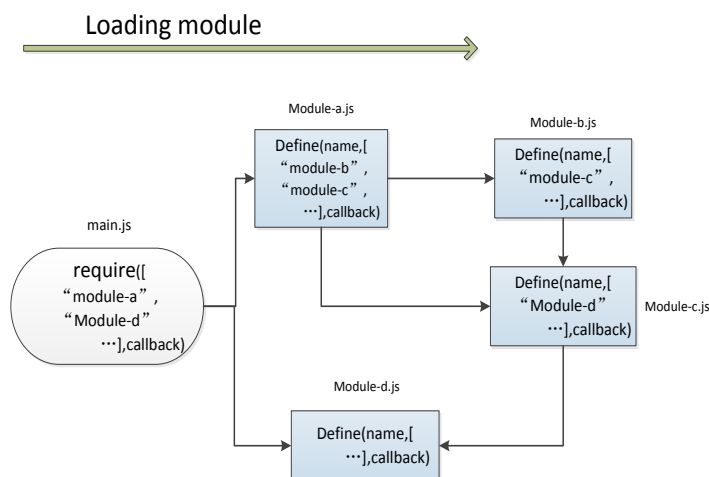


图 4-8 RequireJS 模块加载示意图

RequireJS API 存在于 RequireJS 载入时创建的命名空间 `requirejs` 下。其主要

API 主要是下面三个函数:

- **define** - 该函数用户创建模块。每个模块拥有一个唯一的模块 ID，它被用于 RequireJS 的运行时函数，define 函数是一个全局函数，不需要使用 Requirejs 命名空间。
- **require** - 该函数用于读取依赖。同样它是一个全局函数，不需要使用 Requirejs 命名空间。
- **config** - 该函数用于配置 RequireJS。

异步模块加载模式明确地定义了某个模块所依赖的其他模块，并且只有在他们的依赖项加载完成之后才进行加载，最重要的是异步模块不会触及全局范围，这有效避免了全局变量污染的问题，进而解决了封装性弱和表现效果糟糕等问题。

Cesium 采用异步模块加载机制（AMD）的 RequireJs 来组织底层代码，其基本结构如图 4-9。

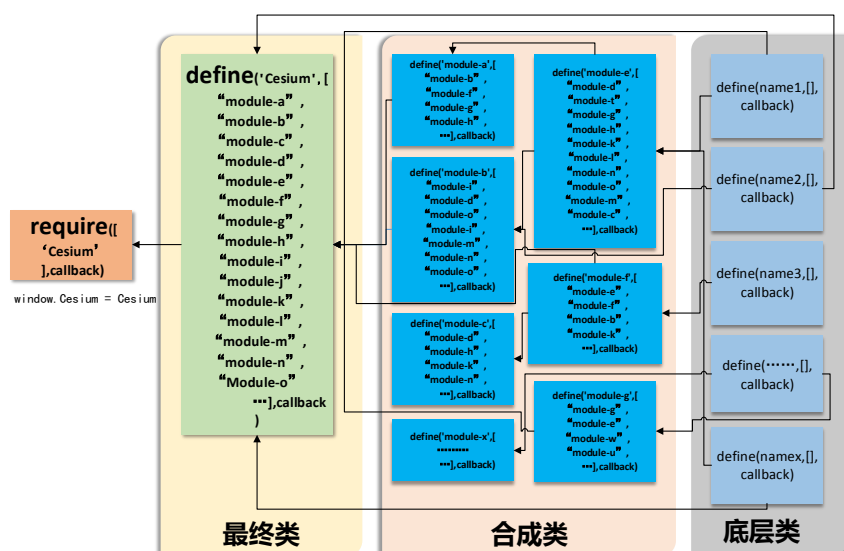


图 4-9 Cesium 以 Requirejs 模块化组织代码示意图

4.1.4 基于 Cesium 的 Web 数字地球原型系统实现

基于前文针对基于 Cesium 的 Web 数字地球原型系统构建方法的阐述，本节利用其中的三种方法，对 Web 数字地球原型系统进行具体构建的实现，作为大规模海洋矢量场动态可视化的展示载体。以下从功能设计、结构设计以及实现效果三方面对该原型系统进行介绍。

1. 功能设计

该基于 Cesium 的 Web 数字地球原型系统主要包括图层管理、常见地理数据支持、量测、查询定位等功能，具体介绍如下：

- (1) **图层管理**: 支持地图的显示功能，地球影像地图默认为 Bing Map 影像数据，同时还提供有 WebMap Service (WMS)、Tile Map Service (TMS)、OpenStreetMap、ArcGIS MapServer 等标准影像，并且支持

影像之间的切换。支持地图漫游功能,在漫游到大比例尺的某一地点时可以借助复位功能到平台初始化地球界面。平台支持地名模糊查询搜索功能。

- (2) **常见地理数据支持:** 由于 Web 平台的局限性,系统开发了针对常见地理数据,如 GeoJSON, TopoJSON, KML, 同时系统也支持 CZML 格式动画数据,并依据 Cesium 平台丰富的 Entity 能够实现丰富的动画效果。系统支持基于 WebGL 的 glTF 格式的三维模型的加载显示,而常见的 dae, 3ds 等格式的模型均可以通过软件转化为 glTF 模型格式。
- (3) **量测功能:** 用户能过量测功能进行对地图上绘图线段与多边形面积的量测,即一种空间面积量测的功能。通过 CesiumMath 类中封装好的方法即可实现。
- (4) **查询及定位功能:** 用户提供查询条件,就能方便的查询到兴趣点的信息,并能在地图上进行定位,也能通过坐标进行定位查询。通过类 Cesium.Cartesian3 中 fromDegrees 方法对经纬度坐标进行转换,并用 flyTo 方法进行定位。
- (5) **标注及画线功能:** 标注与划线功能是 GIS 系统中的最基础功能,通过 Cesium 中的 entities 类来实现。用户可以在地图上进行标注标注与划线功能是 GIS 系统中的最基础的功能,通过 Cesium 中的 entities 类来实现。用户可以在地图上进行标注,并通过贴合地形的画线功能做到三维数据的展示,如图 4 所示。

2. 基本架构设计

由于该 Web 数字地球原型系统主要是作为大规模海洋矢量场动态可视化的展示载体,不包含复杂的地理空间分析功能,只需具备基本的地理信息系统功能即可,结构上比较简单,不需要复杂的结构设计,故只针对 Web 数字地球原型系统的基础功能架构进行设计,具体架构设计图如图 4-10 所示。

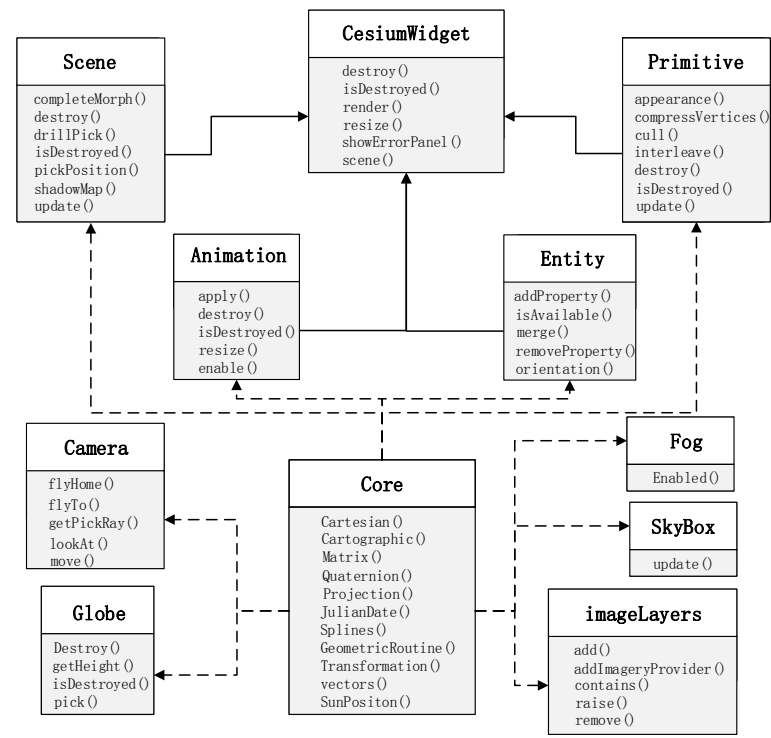


图 4-10 Web 数字地球系统架构设计图

3. Web 数字地球系统效果图

Cesium 封装了 WebGL 复杂的底层实现，这样开发者可以抛弃复杂的 WebGL 原生接口，而不用考虑顶点着色器、片元着色器以及纹理映像等问题。Web 三维开发可以很方便的利用 Cesium 实现一个简单的 Web 数字地球系统，如图 4-11 为一个 Cesium 生成的基本的 Web 数字地球雏形。



图 4-11 基于 WebGL 的虚拟地球雏形 (<http://cesiumjs.org/>)

同时，Cesium 提供了丰富的外部接口来扩展和自定义应用，如图 4-4。开发者可以利用其提供的接口，结合自己的业务应用开发丰富的三维 Web 数字地球应用。如果 Cesium 中未提供相应的接口，开发者也可以在 RequireJs 规范下利用

Cesium 的框架体系开发和定制功能接口,如图 4-12 为数字地球系统理论与系统实验室的“全球环境资源空间信息系统”即是基于此模式开发的 Web 数字地球应用。



图 4-12 全球环境资源空间信息系统 (<http://gers.radi.ac.cn>)

4.2 基于粒子系统的海洋矢量场数据动态可视化

上一节阐述了基于 Cesium 构建 Web 数字地球原型系统方法并予以实现,结合之前介绍的基于粒子系统的海洋矢量场数据动态可视化的方法以及大规模可视化场景的渲染加速方法,本文针对海洋矢量场数据的特点,对其动态可视化进行设计,并进行实验结果的分析。

4.2.1 基于粒子系统的海洋矢量场数据动态可视化设计

本研究以数字地球科学平台中的“全球环境资源空间信息系统”为系统支撑,实现了本文提出的基于粒子系统的海洋矢量场数据动态可视化方法,通过大规模海洋矢量场数据可视化场景的渲染加速方法,实现了大范围的海洋矢量场数据的动态可视化,本文将以海洋风场和海洋洋流场为例。下文从海洋矢量场数据动态可视化模块的功能设计、界面设计和架构设计三个方面进行简要介绍:

1) 功能设计

根据海洋矢量场动态可视化模块所涉及的关键技术和任务要求,从系统开发和软件应用的角度,对相应功能进行了设计。基于 Web 数字地球系统的海洋矢量场动态可视化模块主要包括以下四个功能子模块:

(1) 数据监视和下载: 负责对海洋矢量场数据源的更新进行监视,具体功能包括定期探针侦测,下载更新数据和调用预处理模块等。

(2) NetCDF 预处理和入库：负责对更新的海洋矢量场数据进行预处理以及更新数据，具体功能包括解析 NetCDF，抽取海洋矢量场数据，导出 JSON 数据和更新数据到数据库中等。

(3) 动态可视化规划：负责对海洋矢量场数据的动态可视化进行规划，具体功能包括可视化方法选择，可视化对象选择，场景加速和可视化结果保存等。

(4) 动态可视化的参数配置：负责对海洋矢量场数据的动态可视化的颜色参数、最终渲染方式、动画播放速度等进行设置。

2) 界面设计

为了便于用户的使用和可视化操作，本文根据“全球环境资源空间信息系统”的界面布局以及海洋矢量场动态可视化模块的主要功能，依照简洁、易用、用户友好等 UI 设计原则，设计了海洋矢量场动态可视化模块的使用操作界面。其中可视化操作界面如图 4-13，参数配置界面如图 4-14 所示



图 4-13 启动海洋洋流场可视化模块示意图

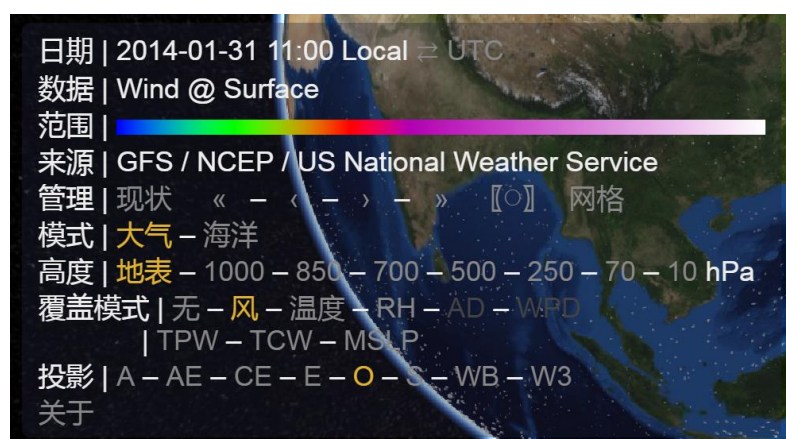
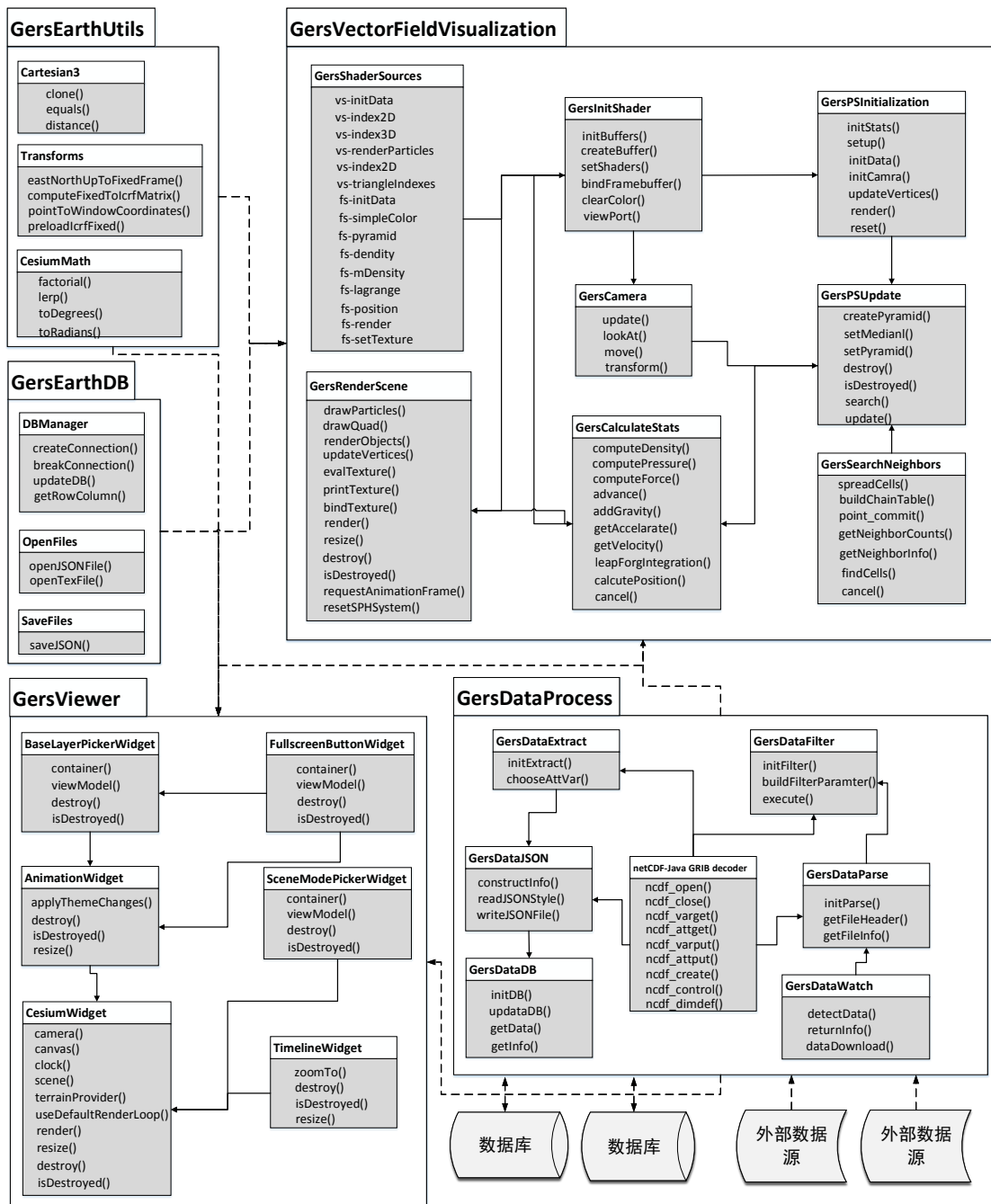


图 4-14 海洋洋流场可视化模块参数配置示意图

3) 架构设计

海洋矢量场动态可视化模块在开发过程中使用了抽象工厂、工厂方法、装饰、



4.2.2 基于粒子系统的海洋矢量场数据动态可视化实现与分析

动态可视化模块进行实现，并给出实现结果的简单对比分析。

1. 可视化实现流程

海洋矢量场可视化模块的具体工作流程主要包括以下四个阶段：

(1) 后台数据监视

该阶段的主要任务是在服务器端针对特定海洋矢量场数据源进行监视，一般由美国国家气象局操作的全球预报系统（GFS）生成，天气与海洋数据由数值模型生成，预测数据每天生成四次，可从 NOMADS (<http://nomads.ncep.noaa.gov/>) 下载。特定的后台数据监视模块监测是否有最新的数据，如果有最新的数据，该模块会下载更新数据和调用预处理模块等。

(2) 数据预处理与入库

该阶段的主要任务是对下载的海洋矢量场数据进行预处理解析，此时主要是 NetCDF，使用 Java 和 NetCDF-Java 类库开发解析 netCDF 的解码器，解析 NetCDF 文件，并提取出所需的海洋矢量场数据，如海洋风场和洋流场数据，最后，导出 JSON 数据，并将其更新数据到数据库中。

(3) 动态可视化参数配置

该阶段的主要任务是对要动态可视化的海洋矢量场要素进行选择，并对可视化的样式进行参数配置，具体来说，就是设置颜色参数、最终渲染方式、动画播放速度等，同时也可以选择可视化的对象，场景加速。

(4) 动态可视化结果

海洋矢量场可视化结果是在动态可视化循环开始运行后，通过可视化手段将可视化效果在显示屏幕上进行三维可视化，同时根据粒子系统的更新情况，展现海洋矢量场的动态变化，并能够根据用户的点击查询该点的实时数据，比如矢量场在该点的大小，方向等。

其整体流程图如下：

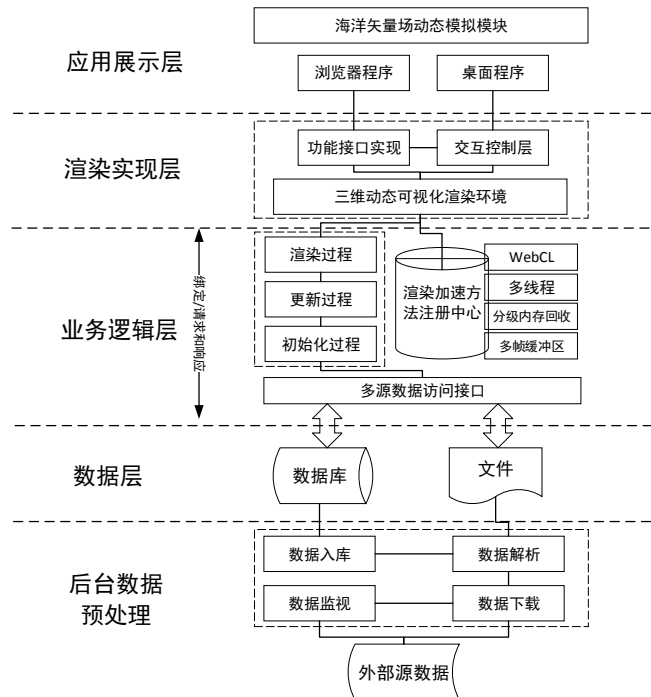


图 4-16 海洋矢量场数据动态可视化流程示意图

2. 实验结果

本文以中国科学院遥感与数字地球研究所自主研发的数字地球科学平台（DESP）的 WebGL 平台“全球环境资源空间信息系统”为系统支撑，实现了本文提出的基于粒子系统的海洋矢量场数据动态可视化方法，通过大规模海洋矢量场数据可视化场景的渲染加速方法，设计和实现了海洋矢量场动态可视化模块。

文中的实验数据由美国国家环境预报中心(NCEP)维护的全球预报系统（GFS）产生，文档是 GRIB2 格式，包含超过 300 条数据记录，天气与海洋数据由数值模型生成，预测每天生成四次，可从 NOMADS（NOAA Operational Model Archive and Distribution System）网站下载。硬件环境为处理器 Inter Core i5 2.7GHz；内存 8GB；硬盘 500GB；显卡 Intel Iris Graphics 6100 1536MB。软件环境为 Google Chrome 57.0 浏览器。本文选取海洋风场数据和海洋流场数据进行可视化，并以多角度对可视化效果进行查看，下面是具体实现效果。

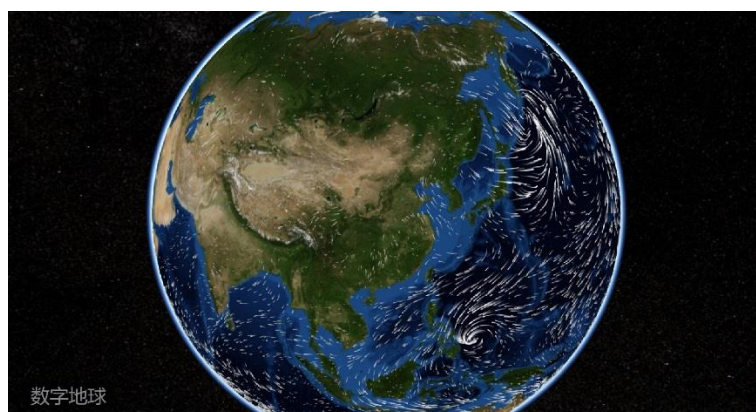


图 4-17 海洋风场全景图

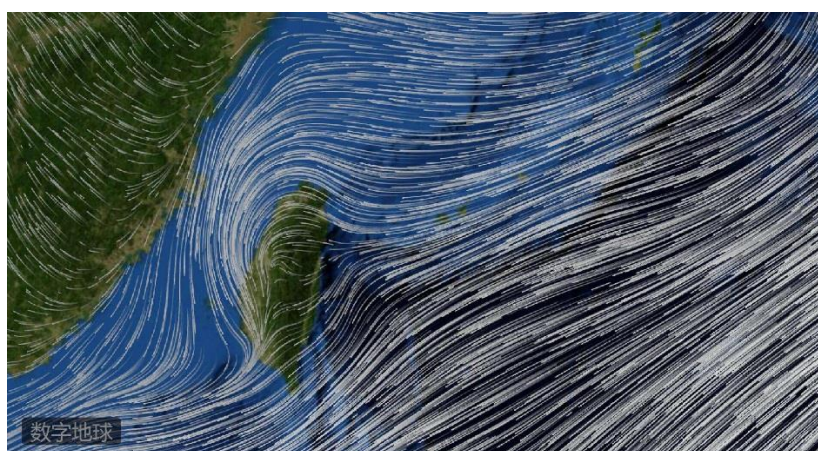


图 4-18 海洋风场俯视图



图 4-19 海洋风场侧视图

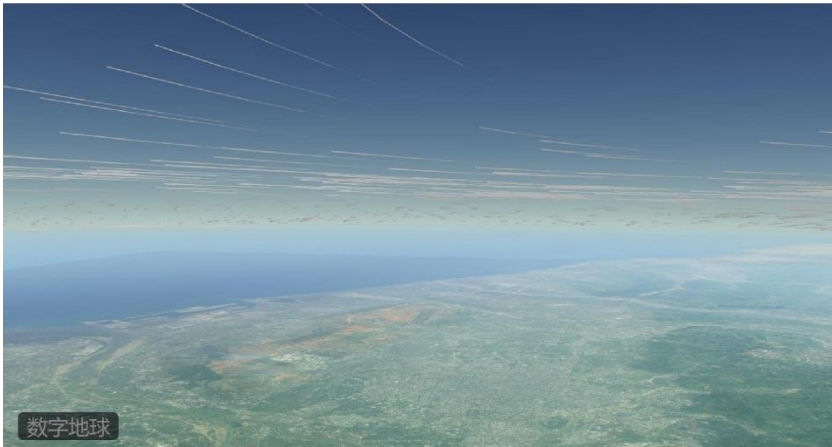


图 4-20 海洋风场仰视图

3. 实验结果分析

点图示法虽然简单，易实现，但存在很多缺陷。采样过密，容易导致图像杂乱无章，显示太少，又不能准确把握矢量场的变化情况，而且该方法难以揭示出数据的内在连续性。矢量线较点图示法有了很大改进，他能表示场的连续性，绘制效果较好，通过初始点的位置适当选取，可以显示矢量场重要的特征和细节。但两者均不能很好的表示矢量场的量的大小和直观的显示矢量场的运动情况。同时，通过可视化效果直观对比，相较于点图标法和矢量线法，本文基于粒子系统提出的方法能够表示场的连续性，能够显示矢量场重要的特征和细节，绘制效果较好，而且，粒子系统法也能够很好的表示矢量场量的大小和直观的显示矢量场的运动情况。

同时，在利用本文提出的方法进行海洋矢量场数据动态可视化时，本文利用之前章节介绍并实现的多种大规模矢量场数据可视化场景的渲染加速方法对整个场景进行渲染加速，包括 WebCL 并行计算、WebWorker 多线程、基于 V8 引擎的分级内存回收算法和多帧缓冲区的 GPU 加速算法。通过多次的加速实验的结果对比分析可清晰看到，在经过多种方式的加速方法协同使用后，整个场景的渲染帧率约由原来的 5 帧约提高了 15 帧，具体的实验对比数据如表 4-1。

表 4-1 整体渲染加速前后对比表（单位：FPS）

实验次数	整体渲染加速前	整体渲染加速后
1	4.96	19.12
2	4.97	19.34
3	4.97	20.04

4	5.12	19.36
5	5.11	20.58
6	4.95	19.72

4.3 本章小结

本章主要从基于 Cesium 的 Web 数字地球原型系统构建和基于粒子系统的海洋矢量场的动态可视化设计与实现两方面来进行论述，其中基于 Cesium 的 Web 数字地球原型系统构建具体阐述了三种逐层深入且灵活的方法来构造个性化的 Web 数字地球系统。同时，在第二部分，本文对海洋矢量场的动态可视化进行了科学和详细的设计，并予以编程实现，最后，对实现结果进行了展示，并进行了简单的对比分析。

5. 结论与展望

5.1 结 论

在地学领域中, 矢量场描述了许多非常重要而且常见的物理现象。矢量场可视化是理解矢量场运动变化规律的有力工具, 在科学计算和工程分析中发挥着重要作用, 具有非常重要的实际意义和研究价值。通过对 WebGL 技术下基于 SPH 进行流体实时模拟的研究, 将改进后的 SPH 运用于海洋矢量场数据的动态可视化, 可直观地揭示多维地学数据间的各种关系及内在规律。本文基于中国科学院遥感与数字地球研究所自主开发的数字地球科学平台, 通过学习国内外相关技术、理论, 研究了海洋矢量场数据动态可视化的相关技术方法, 针对 WebGL 技术下 SPH 实时流体模拟、SPH 原理简化的海洋矢量场动态可视化以及大规模三维可视化场景的渲染加速算法等进行了研究与实现, 取得的主要成果和创新如下:

1. 提出了一种基于 WebGL 的 SPH 实时流体模拟方法

为了在 Web 环境下无插件进行流体实时模拟, 本文提出了一个针对 WebGL 技术的基于多帧缓冲区的 SPH 实时流体模拟具体框架, 分为初始的初始化, 更新和渲染的过程。本文提出的算法比传统的链表搜索算法的渲染帧率快出 5 倍, 同时, 在粒子数目增加至 10 万且进一步增加的情况下, 比经典的 Bayraktar 算法具有明显的优势, 具有很好的实时仿真渲染帧率, 能够满足在 Web 前端进行三维场景渲染帧率的要求。

2. 提出了一种基于粒子系统的海洋矢量场数据动态可视化方法

本文在基于 WebGL 的 SPH 实时流体模拟方法的基础上, 根据海洋矢量场数据的实时特点及其应用范围, 对 SPH 方法的公式进行简化, 并弱化了粒子系统中粒子所受外力的作用, 将算法公式进行重新推导, 最后给出整个算法过程图。最后结合 WebGL、WebWorker 多线程, V8 引擎的分级内存回收等算法对大规模的海洋矢量场数据可视化场景进行渲染加速, 使得整个海洋矢量场数据的动态可视化能够在 Web 环境下流畅的运行。

3. 开发和实现了海洋矢量场动态可视化模块

设计和实现了在 Web 数字地球系统上的海洋矢量场动态可视化模块, 该模块主要包括三个部分, 即服务器端数据处理, 海洋矢量场动态可视化参数配置, 动态可视化的加速渲染, 其中服务器端数据处理是对海洋矢量场源数据进行监视, 如果有最新更新的数据, 调用相关模块对其进行预处理并入库。海洋矢量场动态可视化参数配置实现了对可视化对象和可视化效果展示的相关配置。海洋矢量场

动态可视化的加速渲染是将前文阐述的大规模矢量场数据可视化场景的渲染加速方法运用于海洋矢量场的整体动态可视化,使本文的海洋矢量场动态可视化的帧率由原来的 5 帧提高了 15 帧。

5.2 展 望

通过两年来的研究,本文对海洋矢量场数据动态可视化的研究在广度和深度上都存在一定的局限性,未来的工作应该在以下方面做进一步的深入研究:

1. 粒子系统方法的进一步研究

在利用粒子系统方法对海洋矢量场数据进行可视化方面,虽然通过改进的 SPH 对海洋矢量场数据进行动态可视化,进而能够相对准确直观地显示了海洋矢量场的动态变化趋势,但对于海洋矢量场动态变化规律准确的揭示还稍显乏力。在下一步的海洋矢量场数据可视化工作中,要将更多的海洋矢量场数据的特性考虑进来,同时也要结合实际的海洋环境来改进粒子系统的构建,不能为了可视化而可视化,可视化应该能够更清晰,直观和准确地揭示数据内部的规律。

2. 大规模可视化场景渲染的提高

本文在进行大规模矢量场数据可视化场景渲染的加速方法上,分别阐述了 WebCL, WebWorker 多线程,基于 V8 引擎的分级内存回收算法以及多帧缓冲区的 GPU 加速算法,取得了良好的渲染效果,提高了渲染效率,但由于可视化场景的范围和尺度较大,仍消耗了相当一部分的计算机资源,这在配置相当较低的机器上更为显著。因此在下一步的工作中,应合理分配计算机资源,找出每种方法最适宜的运行时机,进而多种渲染加速方法更加巧妙的结合,充分和合理的利用 CPU 和 GPU 的资源,进一步提高阴影的渲染效率。

3. 真实多因素海洋环境数据的协调

海洋是一个复杂的过程,单一的角度也很难准确反映出物理运行规律,不能通过单一的因素来揭示和反应复杂的海洋运行过程。同时,海洋观测手段的多种多样以及数据获取的频率提高,产生了多种格式的数据,同时数据量也呈井喷式增长,如何对这些数据进行快速的处理和协同式展示与利用是一个值得思考的难题。数据和数据依托的载体以及数据和数据之间,这两组的关系,在可视化的时候需要系统的来思考(以往的多是单纯针对一种要素),在现实中这些都不是孤立而存在的,一个要素的改变肯定会影响到周围环境和要素的改变,甚至可以通过调整构建虚拟海洋环境的参数来影响海洋信息数据的改变。因此,海洋信息的可视化要走向多要素结合集成表达,这也是本文以后的研究方向。

参考文献

- Aaftab Munshi.OpenCL 程序设计指南[M].北京:科学出版社,2012:1-10
- 查石祥. NetCDF数据的程序设计式访问[J].气象科技,2004,32(增刊):40-43.
- 陈英建. SPH实现管道流体实时可视化及在GPU上的优化[D]. 秦皇岛:燕山大学, 2012.
- 丁亮. Navier-Stokes方程SPH的公式解法[J]. 信息技术, 2013, 2(2): 170-174.
- 葛成.三维向量场可视化方法研究[D].南京师范大学,2011.
- 郭秋雷,唐逸之,刘诗秋等. 一个SPH流体实时模拟的全GPU实现框架[J].计算机应用与软件,2011,28(11):69-72.
- 韩冬. 卫星高度计海面风速反演与风场可视化 [D]. 中国海洋大学. 2005.
- 何戡. 复杂流体场景的实时模拟研究[D]. 杭州:浙江大学, 2010.
- 靳超. SPH算法简介(三):光滑核函数[EB/OL]. [2017-2-16]. <http://thecodeway.com/blog/?p=161>.
- 孔倩倩. 基于球面渲染环境的海洋数据多维动态可视化关键技术研究. [D]. 中国海洋大学, 2011.
- Leonwei.从零开始学习OpenCL开发(一) 架构[EB/OL]. [2013-05-03].
<http://blog.csdn.net/leonwei/article/details/8880012>.
- 李骞, 范茵, 王吉奎. 基于粒子追踪的风场可视化方法[J]. 解放军理工大学学报, 2005, 6(1):89-91.
- 李文庆.VV_Ocean海洋环境仿真与海洋数据动态可视化系统的研究与实现[D].中国海洋大学, 2011.
- 林昊. 基于光滑粒子流体动力学的OpenGL可视化[D]. 合肥:安徽大学, 2011.
- 刘健,姜晓轶,范湘涛.海洋环境信息可视化研究进展[J].海洋通报,2014,33(2):235-240.
- 阮一峰. Javascript模块化编程(二): AMD规范.[EB/OL]. [2012-10-30].
http://www.ruanyifeng.com/blog/2012/10/asynchronous_module_definition.html.
- 石鑫祥,雷波.中国数字海洋:理论与实践[M].北京:海洋出版社,2011.
- 宋刚. 基于粒子系统的风场火焰仿真[D]. 青岛大学, 2007.
- 宋丽丽. 面向海量的海洋数据动态可视化与实时绘制技术研究[D].中国海洋大学,2013.
- 唐勇,陈英建,吕梦雅. SPH方法中多维树搜索算法的优化及在GPU上的实现[J].小型微型计算机系统, 2013,34(12):2833-2836.
- 唐泽圣. 三维数据场可视化[M]. 清华大学出版社, 1999.
- 田玲.基于体绘制的三维资料场可视化技术研究[D].西南交通大学,2007.
- 王想红.基于三维虚拟地球的海洋环境数据动态可视化研究[D].辽宁工程技术大学,2013.
- 肖凯涛, 徐路程, 李红辉. 粒子系统在复杂风场及湍流场中的实现[J]. 2015, 39(2):13-21.

- 闫绍博. WebCL拓展模块的设计与实现 [D]. 广州:中山大学, 2015.
- 杨林平. 基于数字地球科学平台的智慧人群行为模拟研究[D]. 中国科学院研究生院. 2013.
- 杨旭波. 基于GPU的可交互实时流体模拟[D]. 上海:上海交通大学, 2014.
- 于海亮, 王彬彬, 吴宛青. 基于粒子追踪技术的海上溢油三维运输模式[J]. 中国航海, 2008, 31(3): 284-288.
- 赵相坤, 李凤霞, 战守义. 基于GPU的面向SPH流体模拟的邻居查找算法[J]. 华南理工大学学报(自然科学版), 2011, 39(7):150-155.
- 周迪斌. 基于纹理的高质量向量可视化研究: [博士学位论文]. 杭州: 浙江大学, 2008.
- 周泉楠, 周贵云. 基于DEM数据的SPH水体流动模拟[J]. 地理空间信息. 2015, 13(2):86-90.
- AI Core. Digital Earth: Understanding our planet in the 21st century[J]. The Australian Surveyor, 1998, 43(2): 89-91.
- Analytical Graphics, Inc. (AGI). Cesium[EP/OL]. [2015-5-12]. <http://cesium.org>.
- ArcGIS Resources[EB/OL]. [2017-4-1]. <http://resources.arcgis.com/zh-cn/help/main/10.1/index.html#na/004600000003000000/>
- Bernd Resch, Ralf Wohlfahrt, Christoph Wosniok. Web-based 4D Visualization of marine geo-data using WebGL. Cartography and Geographic Information Science, Germany. 2014, 41(3):235-247.
- Brian Cabral, Leith Leedom, Imaging Vector Fields Using Line Integral Convolution. Computer Graphics, 1993, 27(4): 263-270
- Craglia M, Bie K, Jackson D, et al. Digital Earth 2020: towards the vision for the next decade[J]. International Journal of Digital Earth, 2012, 5(1): 4-21.
- D Cedeman, P Tsigas. A practical quick sort algorithm for graphics processors[J]. Journal of Experimental Algorithmics, 2009, 14:246-258.
- Dan Bagnell. Graphics Tech in Cesium - Vertex Compression. [EB/OL]. [2016-4-10]. <http://cesiumjs.org/2015/05/18/Vertex-Compression/>.
- Deron Ohlarik. Precisions[M]. 2008.
- Erich Gamma, Richard Helm, Ralph Johnson, et al. Design Patterns[M]. Beijing: China Machine Press, 2000.
- ESRI China. ArcGIS products [EB/OL]. [2015-05-01]. <http://www.esrichina-bj.cn/>.
- EV-Globe[EB/OL]. [2014-2-16]. <http://www.ev-image.com/products/l-3852968750166.html>.
- Fenando Randima. GPU gems: Programming techniques, tips and tricks for real-time graphics[M]. Boston: Addison-Wesley, 2005.
- Goodchild M F, Guo H D, et al. Next-generation Digital Earth[J]. PNAS. 2012, 109(28): 11088-11094.

- Helman J.L, Hesselink L. Surface Representations of Two- and Three-Dimensional Fluid Flow Topology. Proceedings IEEE Visualization 90, 1990: 6-13
- [Http://www.starming.com/index.php?action=plugin&v=wave&tpl=union&ac=viewgroup&gid=34694&tid=12522](http://www.starming.com/index.php?action=plugin&v=wave&tpl=union&ac=viewgroup&gid=34694&tid=12522), 2011-03-31
- Kenwright D, Mallinson G D.A 3D streamline tracking algorithm using dual stream functions. In: Pro Visualization, Boston.1992:62-68.
- Kevin Ring. Rendering the Whole Wide World on the World Wide Web[M]. 2013.
- Khronos Group. WebGL specification[EB/OL]. (2011-2-10).
<http://www.khronos.org/registry/webgl/specs/1.0/>.
- Klokan Technologies. WebGL Earth[CP/OL]. [2015-5-12]. <http://www.webglearth.org>.
- Loesch B, Christen M, Nebiker S. OpenWebGlobe – An Open Source SDK for Creating Large-scale Virtual Globes on a WebGL Basis[C]//XXII ISPRS Congress. International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences. Melbourne, Australia, 2012.
- Lucy L B. A approach to the testing of the fission hypothesis[J]. The Astsion, 1977,8(12):1013-1024.
- Martin Christen, Institute of Geomatics Engineering, University of Applied Sciences, Arts Northwestern Switzerland. Open Web Globe[CP/OL]. [2015-5-12].
<http://www.openwebglobe.org>.
- Matthias Muller, David Charypar, Markus Gross. Particle-Based Fluid Simulation for Interactive Application[J]. ACM SIGGRAPH/Eurographics Symposium on Computer Animation, 2003.
- Micky Kelager. Lagrangian Fluid Dynamics Using Smoothed Particle Hydrodynamics[D]. Copenhagen: University of Copenhagen, 2006.
- Microsoft Inc.Virtual Earth 3D[EB/OL]. [2015-05-01]. <http://baike.baidu.com/view/1640422.htm>.
- Øystein E. Krog, AC Elster. Fast GPU-Based Fluid Simulations Using SPH[J]. Applied Parallel & Scientific Computing-international Conference, 2010, 7134:98-109.
- Patrick Cozzi.Graphics Tech in Cesium - Renderer Architecture[EB/OL]. [2015-05-15].
<http://cesiumjs.org/2015/05/15/Graphics-Tech-in-Cesium-Architecture/>.
- Patrick Cozzi.Graphics-Tech-in-Cesium-Stack.[EB/OL]. [2015-05-26].
<http://cesiumjs.org/2015/05/26/Graphics-Tech-in-Cesium-Stack/>.
- Pharr Matt, Fernando Randima. GPU gems 2: Programming techniques for high-perfonmance graphics and general purpose computation[M]. Boston: Addison-Wesley, 2005.
- REEVES W T, BLAU R. Approximate and probabilistic algorithms for shading and rendering structured particle systems [J]. Computer Graphics, 1985, 19(7): 313-322.

- REEVES W T. Particle systems - a technique for modeling a class of fuzzy objects [J]. In ACM Transactions on Graphics, 1983, 2 (4): 91-108.
- Sakas G. Modeling and Animating 3D Turbulence Using Spectral Synthesis[J]. The Visual Computer, 1993, 9(4):200-211.
- Sim K. Particle Animation and Rendering Using Data Parallel Computation[J]. Computer Graphics, 1990, 24(5):5-12.
- Sloup P. WebGL Earth[D]. Masaryk University Faculty of Informatics, 2011.
- Swrkan Bayraktar, Ugur Gudukbay, Bulent Ozguc. GPU-Based Neighbor-search Algorithm for Particle Simulations[J]. Journal of Graphics GPU & Game Tools, 2009, 14(1):31-42.
- T Harada, S Koshizuka, Y Kawaguchi. Smoothed Particle Hydrodynamics on GPUS[J]. Structure, 2007,4(4):671-691.
- T Harada, S Koshizuka, Y Kawaguchi. Smoothed Particle Hydrodynamics on GPUS[J]. Structure, 2007,4(4):671-691.
- Verma V, Kao D.T, Pang A. A Flow-Guided Streamline Seeding Strategy Proc. IEEE Visualization, 2000: 163-170
- World Wind Resource Center. Open source community portal [EB/OL].
<http://www.worldwindcentral.com>[EB/OL]. 2015/05/01.
- Wu Enhua, Liu Youquan, Liu XueHui. An improved study of real-time fluid simulation on GPU. Journal of Computer Animation and Virtual World. 2004,15(3/4):139-146.
- Xilin Xia, Qiuhua Liang. A GPU-accelerated smoothed Particle Hydrodynamics(SPH) Model for the Shallow Water Equations[J]. Environmental Modelling & Software, 2016, 75:28-43.
- Z Dong, W Chen, H Bao et al. Real-time Voxelization for Complex Models[J]. Computer Graphics & Applications, Pacific Conference, 2015:43-50.

致 谢

时光如梭，在中国科学院遥感与地球研究所的三年研究生生活转瞬即逝。回首那段逝去的时光，不禁让人增添了很多伤感，其中有欢笑，有困惑，有痛苦，也有无数次的彷徨。在这三年中我在科研工作、做人做事等各方面都成长了很多，这都得益于老师、同学、朋友鞭策和帮助以及来自亲人的无私支持，值此硕士学位论文完成之际，特向大家表示衷心的感谢！

首先要感谢的是我的导师范湘涛研究员，范老师为人亲切，平易近人，对学生更是给予充分的科研条件和空间支持，帮助学生成长。范老师学术知识渊博，同时，范老师科研上的大局观让我佩服，在科研的道路上怎样克服课题研究中的难点，第一篇学术论文应该如何写作投稿，大到硕士学位论文应当怎样构思设计，细到论文的章节结构，范老师都给我宝贵的建议。科研工作和生活中，范老师总能及时指出我的不足，有时候的只言片语就能四两拨千斤般直指我的要害，其中蕴含了范老师对学生殷切的期望。在人生最关键的时候，得一良师如此，夫复何求！衷心感谢范老师，向您致以诚挚的敬意。

感谢中国科学院遥感与数字地球研究所数字地球系统研究室的刘朔、杜小平、谭剑、朱俊杰、李京、严珍珍、刘健、詹勤等老师，感谢你们对我学习以及生活中所带来的指导和帮助。感谢我的师兄简洪登、薛铸鑫，郭钊，是你们之前所做的积累才有了我今天的成就，感谢师弟亢叙杰、郭长顺在科研工作生活中给予的支持。感谢实验室的吕其斌、郝俊生、王朋、吕山、刘思齐等所有同事，谢谢你们实验室对时政大事的讨论，让我能够及时接触了很多讯息，我今天的成果离不开你们平日里默默地支持。

感谢研究生处何小伟老师，感谢你为我们的学习和生活保驾护航，是你让我们的研究生生涯更加顺利！

感谢生活中的每一个人，所有同学，朋友，你们的陪伴让我有勇气面对所有的挑战！感谢我的父母、亲人，我的女朋友陈秀妍，感谢你们多年在学业上给予的默默支持，你们是我坚实的后盾，诚挚地祝你们身体健康，一切安好！

最后，感谢自己，感谢自己在科研，工作和生活中遇到困难时咬着牙没有放弃，感谢自己在人生道路上的不断尝试。现在的我撸起袖子加油干，未来的我才会看起来毫不费力。路途漫漫，且行且珍惜！

刘恒星

2017年4月 北京

